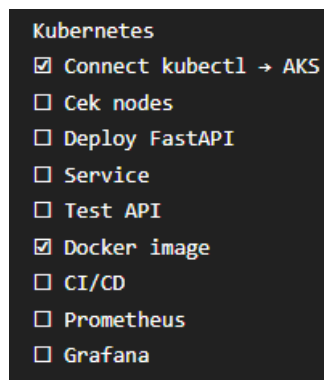


Plan 3



14. Cek nodes

`kubectl get nodes`

```
• (.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform> kubectl get nodes
NAME                                STATUS    ROLES    AGE   VERSION
aks-system-32981788-vmss000000    Ready    <none>   2d1h  v1.35.7
○ (.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform> 
```

15. Deploy FastAPI

Before we deploy, there's one important thing to know: **AKS cannot pull a Docker image that only exists on your laptop.**

So our deployment flow will be:

Docker image on our laptop

↓

Container Registry

↓

AKS

↓

FastAPI Pod

For this project, we'll use **GitHub Container Registry (GHCR)**.

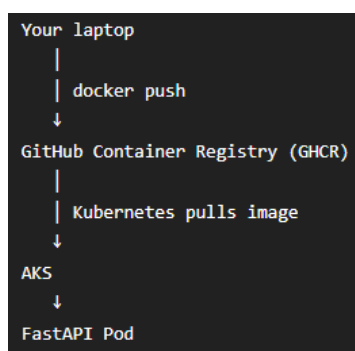
Container Registry is a place to store Docker images so that servers or cloud platforms like AKS can access them. But first, let's do one small step: **check the name of the Docker image currently on your laptop.**

Run:

`docker images`

```
PS C:\WINDOWS\system32> docker images
REPOSITORY          TAG                 IMAGE ID            CREATED             SIZE
task-management-api  latest             34240f2c69c8       40 hours ago       213MB
python               3.12-slim          2f17fc044b57       4 days ago         177MB
gitlab-cicd           latest             d5fc6bfabb4f       7 months ago       192MB
gitlab-cic            latest             6b050f1ed522       7 months ago       192MB
hello-node            latest             e4d6439eb15e       7 months ago       192MB
githubaction-test     latest             897fd359eb07       8 months ago       181MB
my-node-app           latest             4f6c4bf3212c       8 months ago       208MB
studentportal-server  latest             d4152ca93318       9 months ago       1.28GB
mariadb               lts                1cac8492bd78       10 months ago      460MB
nginx                 latest             8adbdc969e2        13 months ago      279MB
hello-world           latest             54e66cc1dd1f       13 months ago      20.3kB
PS C:\WINDOWS\system32> |
```

The step gonna be like this:



15a. Login to GHCR (Github Container Registry)

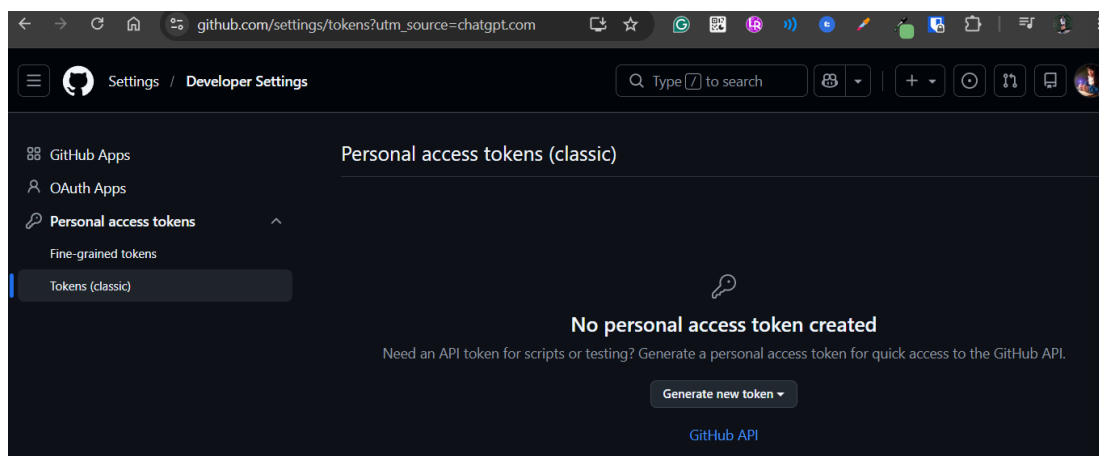
`docker login ghcr.io`

```
Error response from daemon: Get "https://ghcr.io/v2/": denied: denied
PS C:\WINDOWS\system32> docker login ghcr.io
Username: valinfernanda
Password: |
```

For GHCR, you should use a **GitHub Personal Access Token (PAT)** as the password.

link:

https://github.com/login?return_to=https%3A%2F%2Fgithub.com%2Fsettings%2Ftokens%3Futm



So for our project, the token needs at least these permission

☑ write:packages --- to **push/upload** the Docker image to GHCR.

☑ read:packages --- allows you to **pull/download** the Docker image from GHCR.

15b. Tag Image

```
docker tag task-management-api:latest ghcr.io/valinfernanda/task-management-api:latest
```

format : ghcr.io/<github-username>/<image-name>:<tag>

if you're pushing the image to GitHub Container Registry (GHCR), the image should be tagged with the ghcr.io/... registry path before you push it.

replace valinfernanda with your GitHub username.

```
PS C:\WINDOWS\system32> docker tag task-management-api:latest ghcr.io/valinfernanda/task-management-api:latest
PS C:\WINDOWS\system32> docker images
REPOSITORY                                TAG      IMAGE ID      CREATED        SIZE
task-management-api                       latest   34240f2c69c8  41 hours ago  213MB
ghcr.io/valinfernanda/task-management-api latest   34240f2c69c8  41 hours ago  213MB
python                                    3.12-slim 2f17fc044b57  4 days ago    177MB
gitlab-cicd                              latest    d5fc6bfabb4f  7 months ago  192MB
hello-node                               latest    e4d6439eb15e  7 months ago  192MB
gitlab-cic                               latest    6b050f1ed522  7 months ago  192MB
githubaction-test                        latest    897fd359eb07  8 months ago  181MB
my-node-app                              latest    4f6c4bf3212c  8 months ago  208MB
studentportal-server                     latest    d4152ca93318  9 months ago  1.28GB
mariadb                                  lts       1cac8492bd78  10 months ago 460MB
nginx                                     latest    8adbdc969e2   13 months ago 279MB
hello-world                              latest    54e66cc1dd1f  13 months ago 20.3kB
PS C:\WINDOWS\system32> |
```

15c. Push

```
docker push ghcr.io/valinfernanda/task-management-api:latest
```

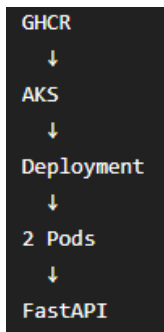
```
PS C:\WINDOWS\system32> docker push ghcr.io/valinfernanda/task-management-api:latest
The push refers to repository [ghcr.io/valinfernanda/task-management-api]
b0dc7f87bef1: Pushed
6b37362b3da7: Pushed
3764a9a7d1e8: Pushed
b3ae8151ac2e: Pushed
06ad939ed42b: Pushed
c03c5daf0732: Pushed
f5cd5a017467: Pushed
be2baebab085: Pushed
d16cbd642074: Pushed
latest: digest: sha256:34240f2c69c8dc05bbe54544e58c4cce664984ac061bc68088c2de345ff70768 size: 856
PS C:\WINDOWS\system32> |
```

Now... the docker image is currently on:

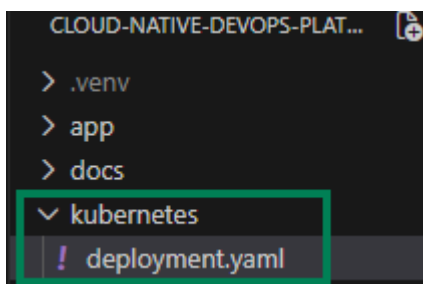
```
GitHub
├─ Container Registry
│   └─ task-management-api
│       └─ latest
```

16. Create Kubernetes Deployment

Now we're going to **actually deploy FastAPI to AKS**.



16a. Create a Kubernetes folder and deployment.yaml



```
apiVersion: apps/v1
```

```
kind: Deployment
```

```
metadata:
```

```
  name: task-management-api
```

```
spec:
```

```
  replicas: 2
```

```
  selector:
```

```
    matchLabels:
```

```
      app: task-management-api
```

```
  template:
```

```
    metadata:
```

```
      labels:
```

```
        app: task-management-api
```

```
    spec:
```

```
      containers:
```

```
        - name: task-management-api
```

```
          image: ghcr.io/valinfernanda/task-management-api:latest
```

```
          imagePullPolicy: Always
```

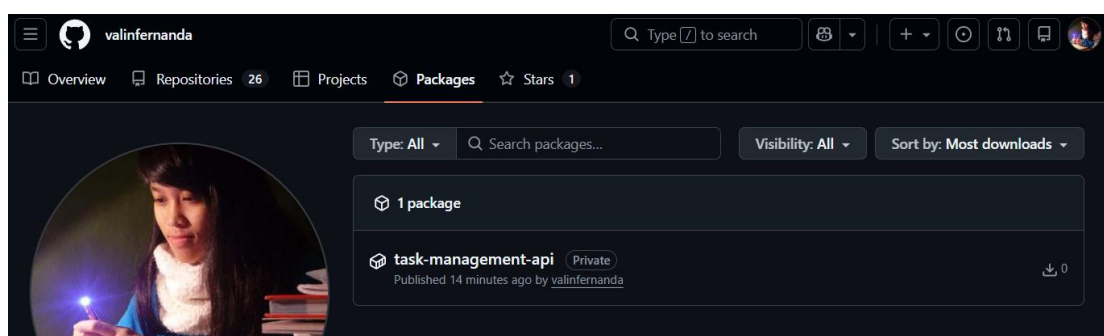
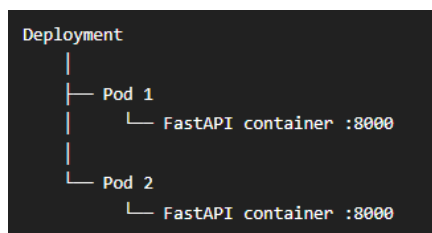
```
      ports:
```

```
        - containerPort: 8000
```

```

kubernetes > ! deployment.yaml > {} spec > {} template > {} spec > {} containers > {} 0 > {} ports > {} 0
io.k8s.api.apps.v1.Deployment (v1@deployment.json)
1  apiVersion: apps/v1
2  kind: Deployment
3
4  metadata:
5    name: task-management-api
6
7  spec:
8    replicas: 2
9
10   selector:
11     matchLabels:
12       app: task-management-api
13
14   template:
15     metadata:
16       labels:
17         app: task-management-api
18
19     spec:
20       containers:
21         - name: task-management-api
22           image: ghcr.io/valinfernanda/task-management-api:latest
23           imagePullPolicy: Always
24
25       ports:
26         - containerPort: 8000
27

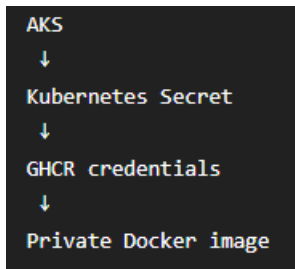
```



If your **GHCR package is public**, AKS can pull the image without any credentials.

If your **GHCR package is private**, AKS needs credentials to access GHCR. We can configure an **imagePullSecret** later.

Because my GHCR is private, I will configure imagePullSecret



16b. Create Kubernetes Secret

Run this in PowerShell:

```
kubectl create secret docker-registry ghcr-secret `
--docker-server=ghcr.io `
--docker-username="YOUR_GITHUB_USERNAME" `
--docker-password="YOUR_GITHUB_TOKEN" `
--docker-email="YOUR_GITHUB_EMAIL"
```

```
kubectl create secret docker-registry ghcr-secret `
--docker-server=ghcr.io `
--docker-username="valinfernanda" `
--docker-password="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" `
--docker-email=valinfernanda08@gmail.com
```

```
PS C:\WINDOWS\system32> kubectl create secret docker-registry ghcr-secret `
>> --docker-server=ghcr.io `
>> --docker-username="valinfernanda" `
>> --docker-password="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" `
>> --docker-email="valinfernanda08@gmail.com"
secret/ghcr-secret created
PS C:\WINDOWS\system32> |
```

16c. Check secret

```
kubectl get secrets
```

```
PS C:\WINDOWS\system32> kubectl get secrets
NAME          TYPE
ghcr-secret   kubernetes.io/dockerconfigjson  1    3m8s
PS C:\WINDOWS\system32> |
```

16d. Connect the Secret to the Deployment

Add this:

```
spec:
  imagePullSecrets:
    - name: ghcr-secret
```

```

kubernetes > ! deployment.yaml > {} spec > {} template > {} spec > {} containers > {} 0 > {} ports > {}
1 apiVersion: apps/v1
2 kind: Deployment
3
4 metadata:
5   name: task-management-api
6
7 spec:
8   replicas: 2
9
10  selector:
11    matchLabels:
12      app: task-management-api
13
14  template:
15    metadata:
16      labels:
17        app: task-management-api
18
19    spec:
20      imagePullSecrets:
21        - name: ghcr-secret
22
23      containers:
24        - name: task-management-api
25          image: ghcr.io/valinfernanda/task-management-api:latest
26          imagePullPolicy: Always
27
28      ports:
29        - containerPort: 8000
30

```

So Kubernetes knows:

“If you need to pull an image from ghcr.io, use the credentials stored in ghcr-secret.”

```

AKS
├── Pod 1
│   └── ghcr.io/valinfernanda/task-management-api:latest
└── Pod 2
    └── ghcr.io/valinfernanda/task-management-api:latest

```

17. Apply Deployment

Make sure you are in the **project root directory**, then run:

```
kubectl apply -f kubernetes/deployment.yaml
```

```

(.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform> kubectl apply -f kubernetes/deployment.yaml
deployment.apps/task-management-api created
(.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform>

```

17a. Check Deployment

```
kubectl get deployments
```

```

(.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform> kubectl get deployments
NAME                READY   UP-TO-DATE   AVAILABLE   AGE
task-management-api  0/2     2            0           79s
(.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform>

```

Ready: 0/2

```
kubectl get pods
```

```
(.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform> kubectl get pods
NAME                                READY   STATUS              RESTARTS   AGE
task-management-api-5965bc845f-2jhcm 0/1     ImagePullBackOff    0           17m
task-management-api-5965bc845f-fr1c7 0/1     ImagePullBackOff    0           17m
(.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform>
```

Status: ImagePullBackoff, means Kubernetes failed to pull the Docker image, then waits for a while before trying again.

And then try to check one of them:

```
kubectl describe pod task-management-api-5965bc845f-2jhcm
```

```
Events:
Type      Reason      Age          From          Message
----      -
Normal    Scheduled   19m         default-scheduler Successfully assigned default/task-management-api-5965bc845f-2jhcm to aks-system-32981788-vmss000000
Normal    Pulling     16m (x5 over 19m) kubelet        Pulling image "ghcr.io/valinfernanda/task-management-api:latest"
Warning   Failed      16m (x5 over 19m) kubelet        Failed to pull image "ghcr.io/valinfernanda/task-management-api:latest": rpc error: code = NotFound desc = get image config descriptor: no match for platform in manifest sha256:34249f2c69c8dc05bbe54544e58c4ccc664984ac061bc68088c2de345ff70768: not found
Warning   Failed      16m (x5 over 19m) kubelet        Error: ErrImagePull
Normal    BackOff     4m27s (x64 over 19m) kubelet        Back-off pulling image "ghcr.io/valinfernanda/task-management-api:latest"
Warning   Failed      4m16s (x65 over 19m) kubelet        Error: ImagePullBackOff
(.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform>
```

No match for platform in manifest, means AKS requests an image with a platform that can run on its node, but the image you pushed to GHCR does not provide that platform.

So, it's better to **check the AKS node's platform directly** instead of guessing.

```
kubectl get nodes -o wide
```

```
(.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform> kubectl get nodes -o wide
NAME                                STATUS   ROLES    AGE    VERSION   INTERNAL-IP   EXTERNAL-IP   OS-IMAGE              KERNEL-VERSION   CONTAINER-RUNTIME
aks-system-32981788-vmss000000      Ready    <none>    2d10h v1.35.7   10.0.1.4      <none>        Ubuntu 24.04.4 LTS    6.8.0-1067-azure  containerd://2.3.3-2
(.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform>
```

So, the issue exists because of this:

```
Kubernetes
↓
"Give me ghcr.io/...:latest"
↓
GHCR
↓
"Image exists"
↓
"Oops, there is no variant compatible with the node's platform"
↓
ImagePullBackOff
```

```
docker manifest inspect ghcr.io/valinfernanda/task-management-api:latest
```

```

(.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform> docker manifest inspect ghcr.io/valinfernanda/task-management-api:latest
{
  "schemaVersion": 2,
  "mediaType": "application/vnd.oci.image.index.v1+json",
  "manifests": [
    {
      "mediaType": "application/vnd.oci.image.manifest.v1+json",
      "size": 1810,
      "digest": "sha256:809a948a7bd430ce9b5fe7451d12b5889dc4e714742ea061d65c82cd06386173",
      "platform": {
        "architecture": "amd64",
        "os": "linux"
      }
    },
    {
      "mediaType": "application/vnd.oci.image.manifest.v1+json",
      "size": 566,
      "digest": "sha256:524ad6f2c91e2ac7ecb7127e569bd6d7e7945e0530d1ee86e62bfa26c46ae9d0",
      "platform": {
        "architecture": "unknown",
        "os": "unknown"
      }
    }
  ]
}

```

Manifest: Information about the image

```
kubectl get nodes -o custom-
```

```
columns=NAME:.metadata.name,ARCH:.status.nodeInfo.architecture
```

```

(.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform> kubectl get nodes -o custom-columns=NAME:.metadata.name,ARCH:.status.nodeInfo.architecture
NAME                                ARCH
aks-system-32981788-vmss000000     arm64
(.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform>

```

There's the problem.

Our AKS node is: aks-system-32981788-vmss000000 **arm64**

Meanwhile, our GHCR image is: linux/**amd64**

So, there is indeed a platform mismatch:

Kubernetes is requesting an image that can run on an arm64 node, but your image does not provide linux/arm64 support.

That is likely what is causing the: **ImagePullBackOff**

Solution

You have **two** options.

Option 1 — Build the image for ARM64

Your AKS node uses arm64, so build the image for ARM64:

```
docker buildx build --platform linux/arm64 `
```

```
-t ghcr.io/valinfernanda/task-management-api:latest `
```

```
--push .
```

Then check the image:

```
docker manifest inspect ghcr.io/valinfernanda/task-management-api:latest
```

You should see:

linux/arm64

Then restart the deployment:

```
kubectl rollout restart deployment task-management-api
```

Option 2 — Build a multi-platform image

This is more flexible because the same image can run on both amd64 and arm64 nodes:

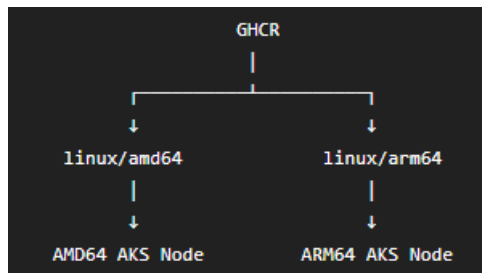
```
docker buildx build --platform linux/amd64,linux/arm64 `
-t ghcr.io/valinfernanda/task-management-api:latest `
--push .
```

The manifest should contain:

linux/amd64

linux/arm64

So the same image can work with both types of AKS nodes:



Option 2 is more flexible because the image supports both platforms.

The unknown/unknown entry is likely an additional image artifact. The important thing is that your image includes **linux/arm64**.

Now... check again

```
docker manifest inspect ghcr.io/valinfernanda/task-management-api:latest
```

```
(.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform\app> docker manifest inspect ghcr.io/valinfernanda/task-management-api:latest
"manifests": [
  {
    "mediaType": "application/vnd.oci.image.manifest.v1+json",
    "size": 1811,
    "digest": "sha256:078d1c136e23f1df975439098431cb5bcf8e1ebafe5b3bde0bda3cd45bcd153",
    "platform": {
      "architecture": "amd64",
      "os": "linux"
    }
  },
  {
    "mediaType": "application/vnd.oci.image.manifest.v1+json",
    "size": 1811,
    "digest": "sha256:80d4c3590ef65a4ce917b9f1ecb409b71cb971ad8b4b9bc7b2547209baa25f8b",
    "platform": {
      "architecture": "arm64",
      "os": "linux"
    }
  }
],
}
```

Mission completed!

Now start from point number 17 again.

kubectl apply -f kubernetes/deployment.yaml

kubectl get deployments

```
(.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform> kubectl apply -f kubernetes/deployment.yaml
deployment.apps/task-management-api unchanged
(.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform> kubectl get deployments
NAME                                READY    UP-TO-DATE    AVAILABLE    AGE
task-management-api                2/2      2              2            93m
(.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform>
```

2/2 READY → both Pods are running and ready.

2 UP-TO-DATE → both Pods use the current Deployment configuration.

2 AVAILABLE → both Pods are available to serve traffic.

We can confirm the pods with: **kubectl get pods -o wide**

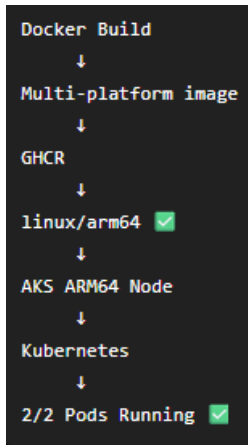
```
(.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform> kubectl get pods -o wide
NAME                                READY    STATUS    RESTARTS    AGE    IP            NODE                                NOMINATED NODE    READINESS GATES
task-management-api-5965bc845f-2jhcm 1/1      Running   0            103m   10.0.1.32     aks-system-32981788-vmss000000    <none>             <none>
task-management-api-5965bc845f-fr1c7 1/1      Running   0            103m   10.0.1.14     aks-system-32981788-vmss000000    <none>             <none>
(.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform>
```

- **2 replicas** were successfully created
- Both containers are **Running**
- Both containers are **Ready**
- RESTARTS = 0 → no crashes
- AKS successfully pulled the private image from **GHCR**
- ghcr-secret is working
- The Pods are running in the AKS subnet 10.0.1.0/24

If you want to make sure the Pods use the newly pushed image, run:

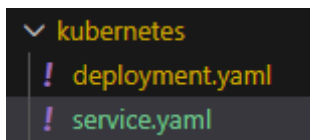
1. `kubectl rollout restart deployment task-management-api`
2. `kubectl rollout status deployment task-management-api`
3. `kubectl get pods -o wide`

So the flow is now working:



18. Creating Kubernetes Service

Update file: `kubernetes/service.yaml`



`apiVersion: v1`

`kind: Service`

`metadata:`

`name: task-management-api`

`spec:`

`selector:`

`app: task-management-api`

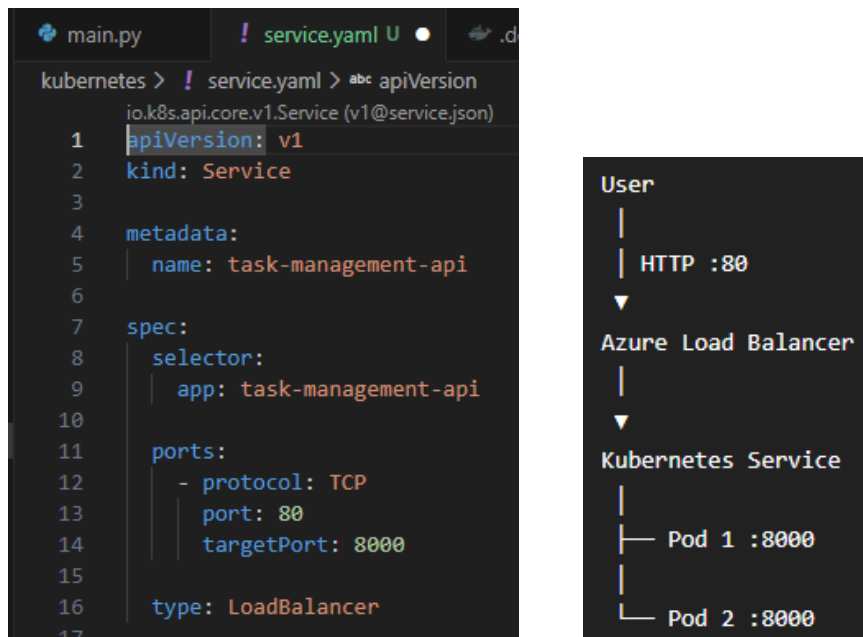
`ports:`

`- protocol: TCP`

`port: 80`

`targetPort: 8000`

`type: LoadBalancer`



We intentionally use a **LoadBalancer** so that Azure creates a **public IP address** for our Service.

port: 80 --- means the **Service receives traffic on port 80**.

targetPort: 8000 --- means: “Once the traffic reaches the Service, forward it to **port 8000 on the Pod**.”

19. Apply the Kubernetes service

```
kubectl apply -f kubernetes/service.yaml
```

```
(.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform> kubectl apply -f kubernetes/service.yaml
service/task-management-api created
(.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform>
```

20. Check Service

```
kubectl get services or kubectl get svc
```

This is what we get when we **do not create a Service** for our application:

```
(.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform> kubectl get services
NAME          TYPE          CLUSTER-IP    EXTERNAL-IP    PORT(S)    AGE
kubernetes    ClusterIP     10.2.0.1      <none>         443/TCP    2d11h
(.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform> kubectl get svc
NAME          TYPE          CLUSTER-IP    EXTERNAL-IP    PORT(S)    AGE
kubernetes    ClusterIP     10.2.0.1      <none>         443/TCP    2d11h
```

Only the default Kubernetes Service exists. There is no Service for task-management-api.

In short: **The Pod is already running without a Service. What's missing is a stable way to access that Pod over the network.**

Each Pod has its own IP address, for example:

- Pod 1 → 10.244.0.10
- Pod 2 → 10.244.0.11

These IP addresses are **internal** and **can change** when a Pod is deleted and recreated.

So if you try to access it like this:

Internet

↓

???

↓

Pod

there is no stable entry point yet.

After Applying Our Service:

This is what we get **after applying the Service we created**:

```
(.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform> kubectl get services
NAME                TYPE          CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE
kubernetes           ClusterIP     10.2.0.1      <none>         443/TCP          2d22h
task-management-api  LoadBalancer 10.2.154.242  172.169.163.204 80:31887/TCP     31s
(.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform>
```

Now, task-management-api has its own **LoadBalancer Service** with an external IP:

172.169.163.204. This service uses a **LoadBalancer** because it needs to be accessible from the Internet, while the backend-service only needs a **ClusterIP** because it is accessed by other applications within the cluster.

80 → port Service

31887 → NodePort yang Kubernetes gunakan untuk meneruskan traffic melalui node

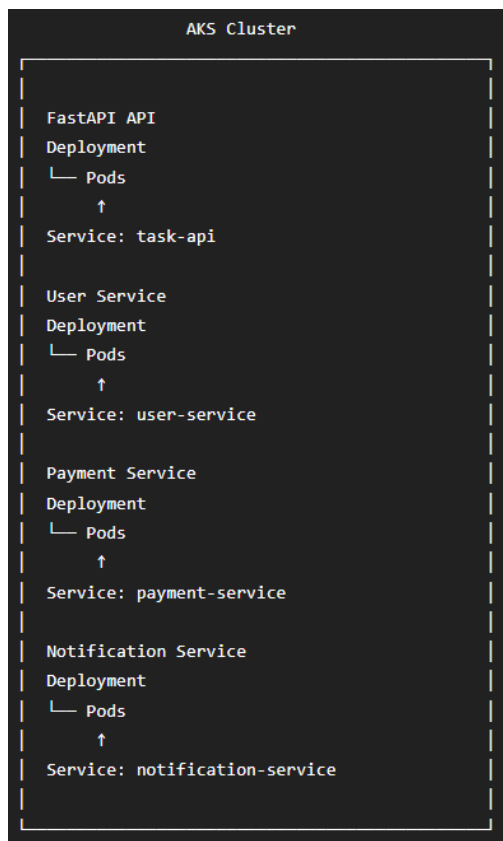
TCP → protocol

So, a single **AKS cluster can have multiple Services**. In larger projects, you can even have **dozens or hundreds of Services** within one cluster.

What differentiates them is **the function of each Service**, not the cluster itself.

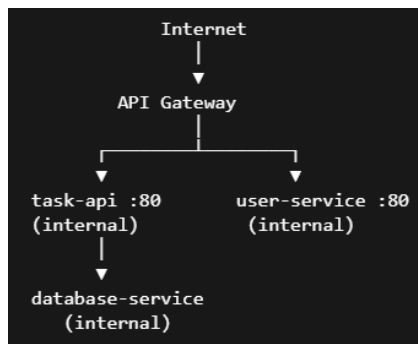
You can have multiple microservices within a single Kubernetes cluster, and each microservice typically has its own **Deployment + Service**.

Example:



What's interesting is that **not every Service needs to be public**.

For example:



You can have:

- **LoadBalancer** → for Services that need to be accessed from the Internet.
- **ClusterIP** → for internal communication between microservices.
- **NodePort** → typically used for specific use cases, rather than being the primary choice for production.

So, for example, **only the API Gateway might be public**, while:

- user-service

- payment-service
- notification-service
- database

remain **private inside the cluster**.

Deployment = runs the application

Service = provides network access to the application

Cluster = the environment where all these components live

kubectl get pods

```

(.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform> kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
task-management-api-5965bc845f-2jhcm 1/1     Running   0           17h
task-management-api-5965bc845f-frlc7 1/1     Running   0           17h
(.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform>

```

kubectl get services

```

(.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform> kubectl get services
NAME            TYPE          CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
kubernetes      ClusterIP     10.2.0.1     <none>        443/TCP          3d3h
task-management-api LoadBalancer 10.2.154.242 172.169.163.204 80:31887/TCP    4h42m

```

Record the external IP and then open **http://EXTERNAL-IP/docs**

or if you wanna try it out by PowerShell, can use **curl http://EXTERNAL-IP/docs**

The screenshot shows a web browser window with the address bar displaying '172.169.163.204/docs'. The page title is 'Task Management API' with version indicators '0.1.0' and 'OAS 3.1'. Below the title, there is a 'default' section with a list of API endpoints:

- GET / Root
- GET /health Health
- GET /tasks Get Tasks
- POST /tasks Create Task

To the right of the browser window, there is a vertical diagram showing the architecture stack:

```

graph TD
    User --> PublicIP[Public IP]
    PublicIP --> AzureLoadBalancer[Azure Load Balancer]
    AzureLoadBalancer --> KubernetesService[Kubernetes Service]
    KubernetesService --> Pod
    Pod --> DockerContainer[Docker Container]
    DockerContainer --> FastAPI

```

Now, our AKS deployment is **successfully** working **end-to-end**. Yaaa!

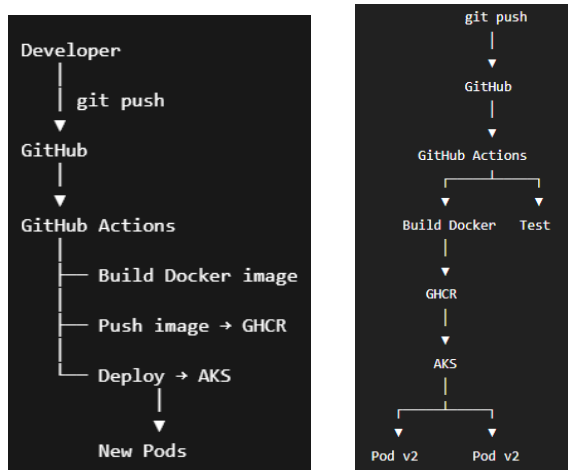
Our Milestone So Far

- Azure infrastructure
- Terraform
- AKS
- Docker image
- GHCR
- ghcr-secret
- Kubernetes Deployment
- 2 replicas
- Kubernetes Service
- Public access
- FastAPI accessible from the Internet

Next: CI/CD

Now we get to one of the most interesting parts.

We're going to build this workflow:



So from now on, we **won't need to manually run:**

```
docker build
docker push
kubectl apply
```

Every time you push a change to GitHub, **the CI/CD pipeline will automatically build the Docker image, push it to GHCR, and deploy the new version to AKS.**

21. Create the Workflow

In our project, create the following folders:

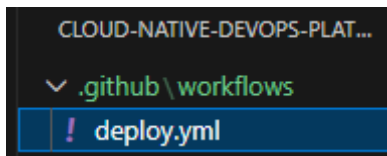
```
mkdir .github
```

```
mkdir .github\workflows
```

Then create this file:

```
.github/workflows/deploy.yml
```

For now, leave the file empty. (we will continue this on the step number 22)



22. GitHub Authentication

GitHub Actions needs authentication for **two things**:

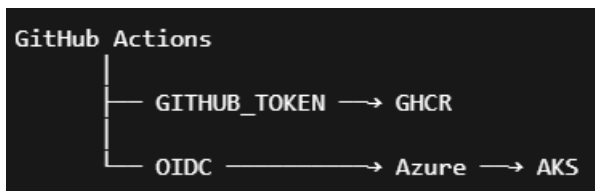
1. Push the Docker image to GHCR

- We can use the built-in GITHUB_TOKEN.
- No need to create a GitHub Personal Access Token.

2. Login to Azure and deploy to AKS

- We'll use **OIDC / federated identity**.
- This is more secure and modern than storing an Azure password or service principal secret.

In simple terms:



So we don't need to store regular passwords or GitHub tokens in the workflow.

OIDC (OpenID Connect) is a way for **GitHub Actions to authenticate with Azure without storing an Azure password or secret.**

21a. Need to know the information of our Azure Project

```
az aks list -o table
```

```
(.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform> az aks list -o table
Name                                Location    ResourceGroup    KubernetesVersion    CurrentKubernetesVersion    ProvisioningState    Fqdn
-----
aks-devops-platform                centralus   rg-devops-platform    1.35                  1.35.7                      Succeeded            aks-devops-platform-pl1bpg1l.hcp.centralus.azmk8s.io
```

```
az aks show `
```

```
--resource-group <RESOURCE_GROUP_NAME> `
```

```
--name <AKS_NAME> `
```

```
--query "{name:name, resourceGroup:resourceGroup, subscriptionId:id}" `
```

```
-o table
```

```
az aks show `
```

```
--resource-group rg-devops-platform `
```

```
--name aks-devops-platform `
```

```
--query "{name:name, resourceGroup:resourceGroup, subscriptionId:id}" `
```

```
-o table
```

```
(.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform> az aks show `
>> --resource-group rg-devops-platform `
>> --name aks-devops-platform `
>> --query "{name:name, resourceGroup:resourceGroup, subscriptionId:id}" `
>> -o table
Name                                ResourceGroup    SubscriptionId
-----
aks-devops-platform                rg-devops-platform    /subscriptions/00e7e36a-186f-4d39-a117-ab05fbd6b847/resourcegroups/rg-devops-platform/providers/Microsoft.ContainerService/managedClusters/aks-devops-plat
form
```

Reminder:

Resource Group is like a **container in Azure** used to group related resources together.

AKS Name is the **name of the Kubernetes cluster**.

```
az account show
```

```
(.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform> az account show
{
  "environmentName": "AzureCloud",
  "homeTenantId": "62ab525c-f89b-49ac-94e1-257f270456d4",
  "id": "00e7e36a-186f-4d39-a117-ab05fbd6b847",
  "isDefault": true,
  "managedByTenants": [],
  "name": "Azure subscription 1",
  "state": "Enabled",
  "tenantDefaultDomain": "valinfernanda08gmail.onmicrosoft.com",
  "tenantDisplayName": "Default Directory",
  "tenantId": "62ab525c-f89b-49ac-94e1-257f270456d4",
  "user": {
    "name": "valinfernanda08@gmail.com",
    "type": "user"
  }
}
```

Now, we already have all the identifiers we need.

The important ones are:

Subscription ID : 00e7e36a-...

Tenant ID : 62ab525c-...

Resource Group : rg-devops-platform

AKS : aks-devops-platform

Note: Subscription ID and Tenant ID are identifiers, not passwords or secrets, but it's still a good practice to avoid sharing them unnecessarily.

21b. Create an Azure User-Assigned Managed Identity

Why should Managed Identity? To **give GitHub Actions an identity in Azure**, so GitHub Actions can deploy to Azure/AKS **without storing an Azure password or secret**.

This is necessary because **GitHub Actions is not a regular Azure user**.

You can find the explanation in the table below:

	Regular User	GitHub Actions
Identity	Entra ID user	Managed Identity
Login	az login	OIDC
Long-term Credential	Not required for interactive login	Not required
Permissions	Azure RBAC	Azure RBAC
Purpose	You manage Azure	Automation/CI-CD manages Azure

```
az identity create `
```

```
--name github-actions-aks `
```

```
--resource-group rg-devops-platform
```

```

(.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform> az identity create `
>> --name github-actions-aks `
>> --resource-group rg-devops-platform
{
  "assignmentRestrictions": {
    "providers": []
  },
  "clientId": "e5e6c957-a62d-4cde-b877-35bbbc3412c6",
  "id": "/subscriptions/00e7e36a-186f-4d39-a117-ab05fbd6b847/resourcegroups/rg-devops-platform/providers/Microsoft.ManagedIdentity/userAssignedIdentities/github-actions-aks",
  "isolationScope": "None",
  "location": "centralus",
  "name": "github-actions-aks",
  "principalId": "0b7b04fa-3ebf-4348-955d-1dab49caf2a0",
  "resourceGroup": "rg-devops-platform",
  "tags": {},
  "tenantId": "62ab525c-f80b-49ac-94e1-257f270456d4",
  "type": "Microsoft.ManagedIdentity/userAssignedIdentities"
}

```

Take note of the **ClientID** and **principalID** above. To keep things clean, use this:

a. clientId

```
az identity show `
```

```
--name github-actions-aks `
```

```
--resource-group rg-devops-platform `
```

```
--query clientId `
```

```
-o tsv
```

b. principalID

```
az identity show `
```

```
--name github-actions-aks `
```

```
--resource-group rg-devops-platform `
```

```
--query principalId `
```

```
-o tsv
```

```

(.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform> az identity show `
>> --name github-actions-aks `
>> --resource-group rg-devops-platform `
>> --query clientId `
>> -o tsv
e5e6c957-a62d-4cde-b877-35bbbc3412c6
(.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform> az identity show `
>> --name github-actions-aks `
>> --resource-group rg-devops-platform `
>> --query principalId `
>> -o tsv
0b7b04fa-3ebf-4348-955d-1dab49caf2a0

```

clientId : e5e6c957-a62d-4cde-b877-35bbbc3412c6

principalID : 0b7b04fa-3ebf-4348-955d-1dab49caf2a0

21c. Assign Role

We will assign the **Azure Kubernetes Service Cluster User Role** to the identity on the AKS cluster.

Notes: we need principalID

```
az role assignment create `
```

```
--assignee <PRINCIPAL_ID> `
```

```
--role "Azure Kubernetes Service Cluster User Role" `
```

```
--scope "/subscriptions/00e7e36a-186f-4d39-a117-ab05fbd6b847/resourceGroups/rg-devops-  
platform/providers/Microsoft.ContainerService/managedClusters/aks-devops-  
platform"
```

```
(.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform> az role assignment create `
>> --assignee 0b7b04fa-3ebf-4348-955d-1dab49caf2a0 `
>> --role "Azure Kubernetes Service Cluster User Role" `
>> --scope "/subscriptions/00e7e36a-186f-4d39-a117-ab05fbd6b847/resourceGroups/rg-devops-platform/providers/Microsoft.ContainerService/managedClusters/aks-devops-platform"
{
  "condition": null,
  "conditionVersion": null,
  "createdBy": null,
  "createdOn": "2026-09-24T19:30:36.382698+00:00",
  "delegatedManagedIdentityResourceId": null,
  "description": null,
  "id": "/subscriptions/00e7e36a-186f-4d39-a117-ab05fbd6b847/resourceGroups/rg-devops-platform/providers/Microsoft.ContainerService/managedClusters/aks-devops-platform/providers/Microsoft.Authorization/roleAssignments/3b097de4-2c7e-4ae6-81ee-6e59820e7677",
  "name": "3b097de4-2c7e-4ae6-81ee-6e59820e7677",
  "principalId": "0b7b04fa-3ebf-4348-955d-1dab49caf2a0",
  "principalType": "ServicePrincipal",
  "resourceGroup": "rg-devops-platform",
  "roleDefinitionId": "/subscriptions/00e7e36a-186f-4d39-a117-ab05fbd6b847/providers/Microsoft.Authorization/roleDefinitions/4abcc35-e782-43d8-92c5-2d3f1bd2253f",
  "scope": "/subscriptions/00e7e36a-186f-4d39-a117-ab05fbd6b847/resourceGroups/rg-devops-platform/providers/Microsoft.ContainerService/managedClusters/aks-devops-platform",
  "systemData": null,
  "type": "Microsoft.Authorization/roleAssignments",
  "updatedBy": "0a6712dc-9df7-458b-8382-db8c21b248c",
  "updatedOn": "2026-09-24T19:30:36.940708+00:00"
}
```

Create Managed Identity



Identity is created



Assign a Role



Identity has permission to access AKS

21d. Federated Credentials and OIDC (for OIDC, we will do it in the next step, in the deploy.yml)

OIDC is a way for **GitHub Actions** to prove its identity to **Azure**.

Because OIDC should be restricted to a **specific repository**, Azure needs to know the Github Repository.

git remote -v

```

• (.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform> git remote -v
origin https://github.com/valinfernanda/cloud-native-devops-platform.git (fetch)
origin https://github.com/valinfernanda/cloud-native-devops-platform.git (push)

```

GitHub owner: valinfernanda

Repository: cloud-native-devops-platform

Next, create a **Federated Credential**

A **Federated Credential** answers: “Which GitHub Actions workflows are allowed to use this Managed Identity?”

```
az identity federated-credential create `
```

```
--name github-actions `
```

```
--identity-name github-actions-aks `
```

```
--resource-group rg-devops-platform `
```

```
--issuer "https://token.actions.githubusercontent.com" `
```

```
--subject "repo:valinfernanda/cloud-native-devops-
platform:ref:refs/heads/main" `
```

```
--audiences "api://AzureADTokenExchange"
```

```

• (.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform> az identity federated-credential create `
> --name github-actions `
> --identity-name github-actions-aks `
> --resource-group rg-devops-platform `
> --issuer "https://token.actions.githubusercontent.com" `
> --subject "repo:valinfernanda/cloud-native-devops-platform:ref:refs/heads/main" `
> --audiences "api://AzureADTokenExchange"
{
  "audiences": [
    "api://AzureADTokenExchange"
  ],
  "id": "/subscriptions/00e7e36a-186f-4d39-a117-ab05fbd6b847/resourcegroups/rg-devops-platform/providers/Microsoft.ManagedIdentity/userAssignedIdentities/github-actions-aks/federatedIdentityCredentials/github-actions",
  "issuer": "https://token.actions.githubusercontent.com",
  "name": "github-actions",
  "resourceGroup": "rg-devops-platform",
  "subject": "repo:valinfernanda/cloud-native-devops-platform:ref:refs/heads/main",
  "type": "Microsoft.ManagedIdentity/userAssignedIdentities/federatedIdentityCredentials"
}

```

What We Are Actually Telling Azure

repo:valinfernanda/cloud-native-devops-platform:ref:refs/heads/main

means:

"I only trust GitHub Actions coming from the valinfernanda/cloud-native-devops-platform repository and the **main branch."**

One Thing to Check: Make sure the main branch of your repository is actually **main**. `git branch --show-current`

If the output is main, then you are good.

```

• (.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform> git branch --show-current
main

```

22. Create deploy.yml

```
cloud-native-devops-platform/  
└─ .github/  
    └─ workflows/  
        └─ deploy.yml
```

name: Build and Deploy

on:
 push:
 branches:
 - main

permissions:
 id-token: write
 contents: read

env:
 IMAGE_NAME: ghcr.io/valinfernanda/task-management-api
 RESOURCE_GROUP: rg-devops-platform
 AKS_CLUSTER: aks-devops-platform

jobs:
 build-and-deploy:

runs-on: ubuntu-latest

steps:

- name: Checkout repository
 uses: actions/checkout@v4

- name: Login to GHCR
 uses: docker/login-action@v3
 with:
 registry: ghcr.io
 username: \${{ github.actor }}
 password: \${{ secrets.GITHUB_TOKEN }}

- name: Build Docker image
 run: |
 docker build \
 -t \${{ env.IMAGE_NAME }}:\${{ github.sha }} \
 -t \${{ env.IMAGE_NAME }}:latest \
 ./app

- name: Push Docker image

```
run: |  
  docker push ${env.IMAGE_NAME}:${github.sha}  
  docker push ${env.IMAGE_NAME}:latest
```

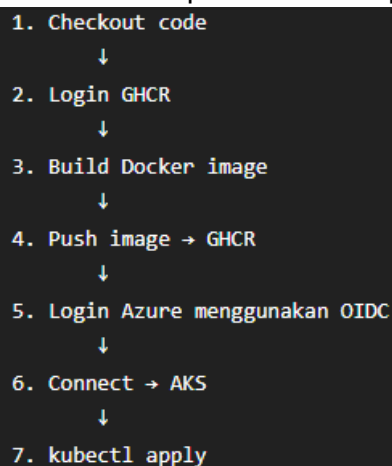
```
- name: Azure Login  
  uses: azure/login@v2  
  with:  
    client-id: ${secrets.AZURE_CLIENT_ID}  
    tenant-id: ${secrets.AZURE_TENANT_ID}  
    subscription-id: ${secrets.AZURE_SUBSCRIPTION_ID}
```

```
- name: Set AKS context  
  uses: azure/aks-set-context@v4  
  with:  
    resource-group: ${env.RESOURCE_GROUP}  
    cluster-name: ${env.AKS_CLUSTER}
```

```
- name: Deploy to AKS  
  run: |  
    kubectl apply -f k8s/deployment.yaml  
    kubectl apply -f k8s/service.yaml
```

```
- name: Verify deployment  
  run: |  
    kubectl rollout status deployment/task-management-api  
    kubectl get pods  
    kubectl get service task-management-api
```

This workflow performs **7 steps**:



There is one very important part:

```
-t ${env.IMAGE_NAME}:${github.sha}
```

For example, if your commit is: a81f92c...

the image becomes: ghcr.io/valinfernanda/task-management-api:a81f92c...

We also push: **:latest**

So GHCR will have:

```
task-management-api
├─ latest
├─ a81f92c
├─ b72e31a
└─ ...
```

This is good because every build has an immutable, commit-based tag, rather than relying only on latest.

But...don't push yet!

The workflow above requires **3 GitHub Actions Secrets**:

AZURE_CLIENT_ID

AZURE_TENANT_ID

AZURE_SUBSCRIPTION_ID

a. AZURE_TENANT_ID

```
az account show --query tenantId -o tsv
```

```
(.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform> az account show --query tenantId -o tsv
62ab525c-f89b-49ac-94e1-257f270456d4
```

62ab525c-f89b-49ac-94e1-257f270456d4

b. AZURE_SUBSCRIPTION_ID

```
az account show --query id -o tsv
```

```
(.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform> az account show --query id -o tsv
00e7e36a-186f-4d39-a117-ab05fbd6b847
```

00e7e36a-186f-4d39-a117-ab05fbd6b847

c. AZURE_CLIENT_ID

```
az identity show `
```

```
--name github-actions-aks `
```

```
--resource-group rg-devops-platform `
```

```
--query clientId `
```

```
-o tsv
```

```

● (.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform> az identity show `
>> --name github-actions-aks `
>> --resource-group rg-devops-platform `
>> --query clientId `
>> -o tsv
e5e6c957-a62d-4cde-b877-35bbbc3412c6

```

e5e6c957-a62d-4cde-b877-35bbbc3412c6

Notes: Do not put these values directly into deploy.yml

Instead, we'll add them here:

GitHub → Repository → Settings → Secrets and variables → Actions

23. Commit workflow

In PowerShell, run `git status`

We should see something like:

```

modified:   ...
new file:   .github/workflows/deploy.yml

```

This lets us check which files have been **modified or newly created** before we commit and push the changes.

Now, let us commit the workflow :

```
git add .github/workflows/deploy.yml
```

```
git commit -m "Add GitHub Actions CI/CD pipeline"
```

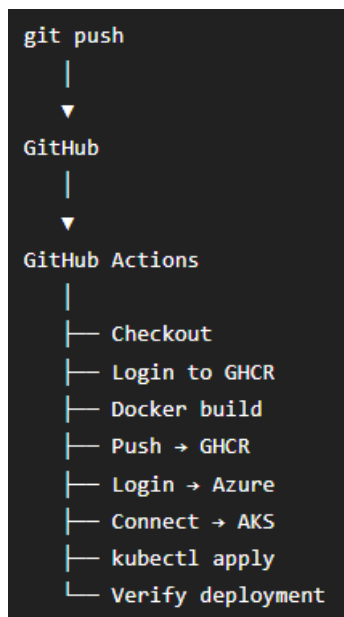
```
git push origin main
```

```

● (.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform> git add .github/workflows/deploy.yml
● (.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform> git commit -m "Add GitHub Actions CI/CD pipeline"
[main ace1419] Add GitHub Actions CI/CD pipeline
1 file changed, 68 insertions(+)
create mode 100644 .github/workflows/deploy.yml
● (.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform> git push origin main
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 12 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (5/5), 1.00 KiB | 514.00 KiB/s, done.
Total 5 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/valinfernanda/cloud-native-devops-platform.git
e1a2ded..ace1419 main -> main

```

The flow now becomes:



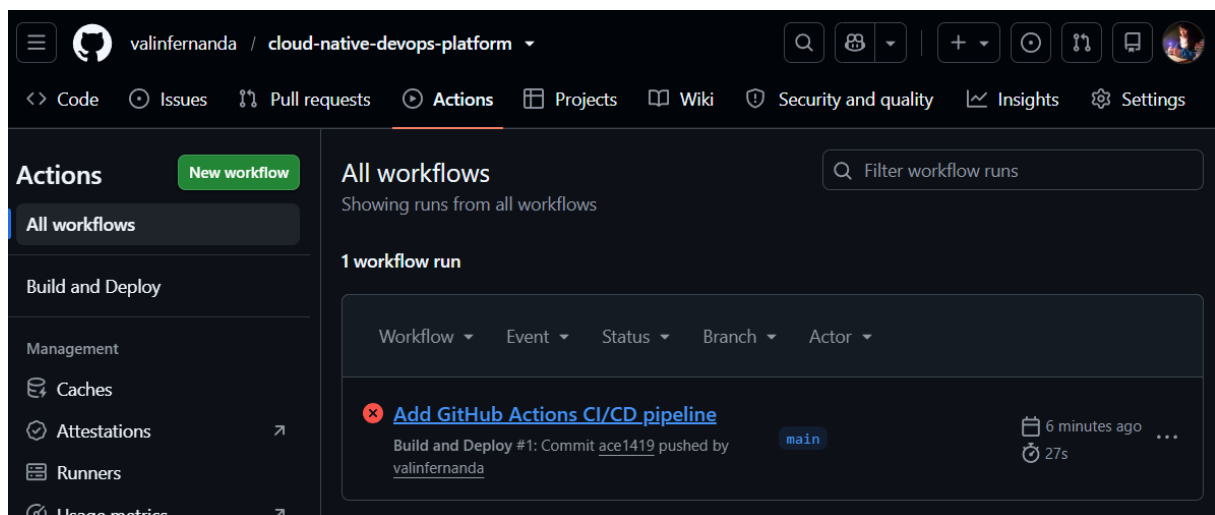
24. Check the Pipeline

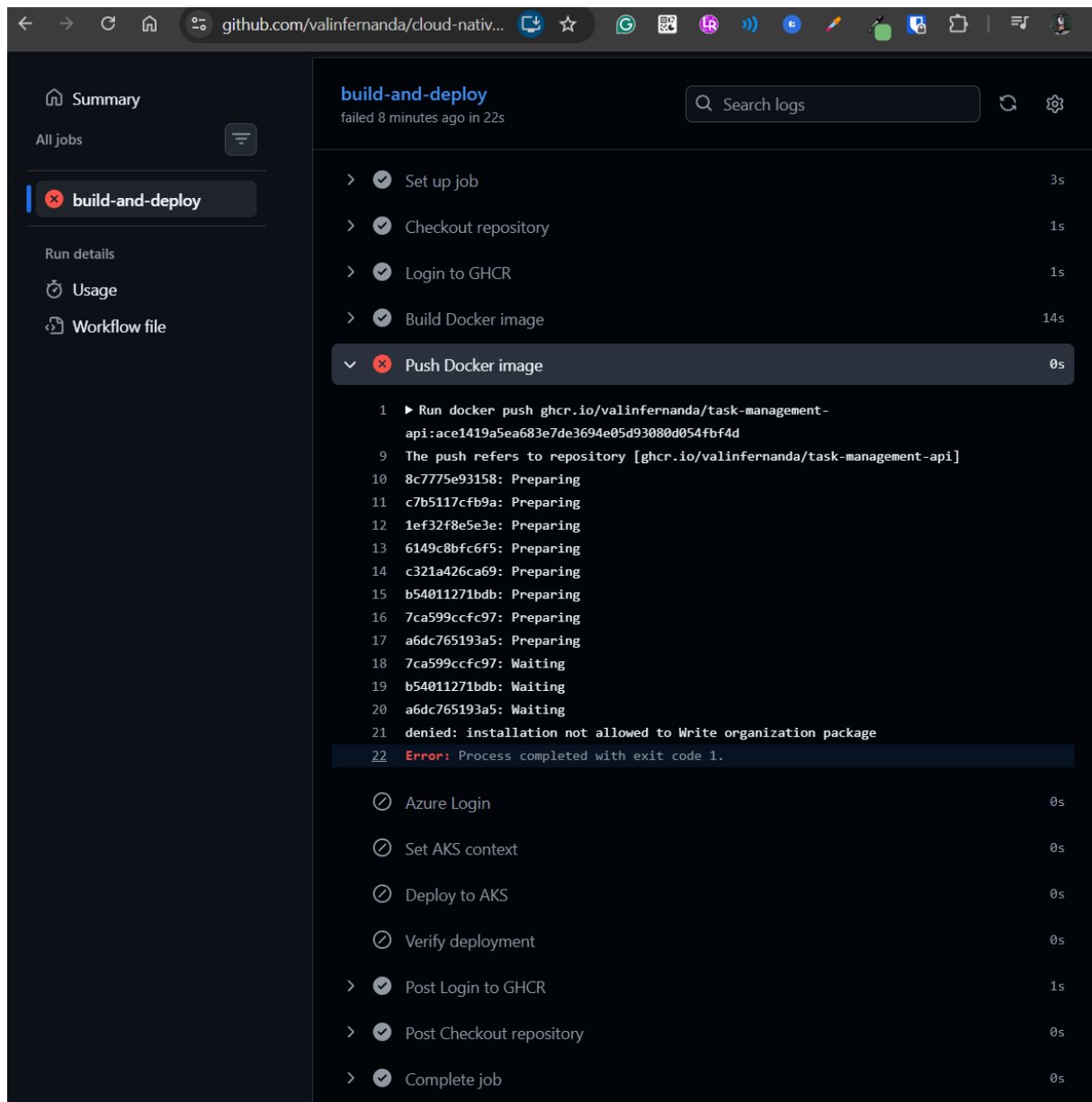
Open your GitHub repository → go to the:

Actions tab

You should see: Build and Deploy

Click the workflow and check each step.





The main error is:

denied: installation not allowed to Write organization package

Why does this happen?

My workflow is trying to:

GitHub Actions

↓

docker push

↓

GHCR

↓

Write access to the package is not allowed

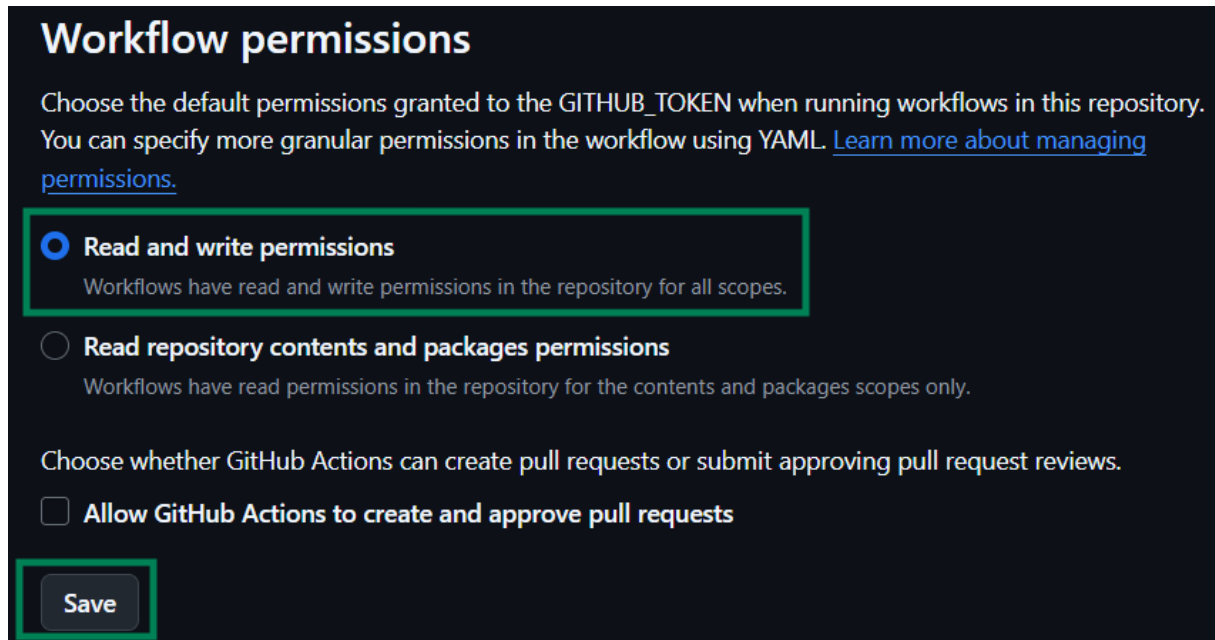
Most likely, the repository/package is under an organization, or the GitHub App/Actions installation **does not have permission to write packages**.

What should we check first?

In your GitHub repository: **Settings** → **Actions** → **General**

Look for: **Workflow permissions**

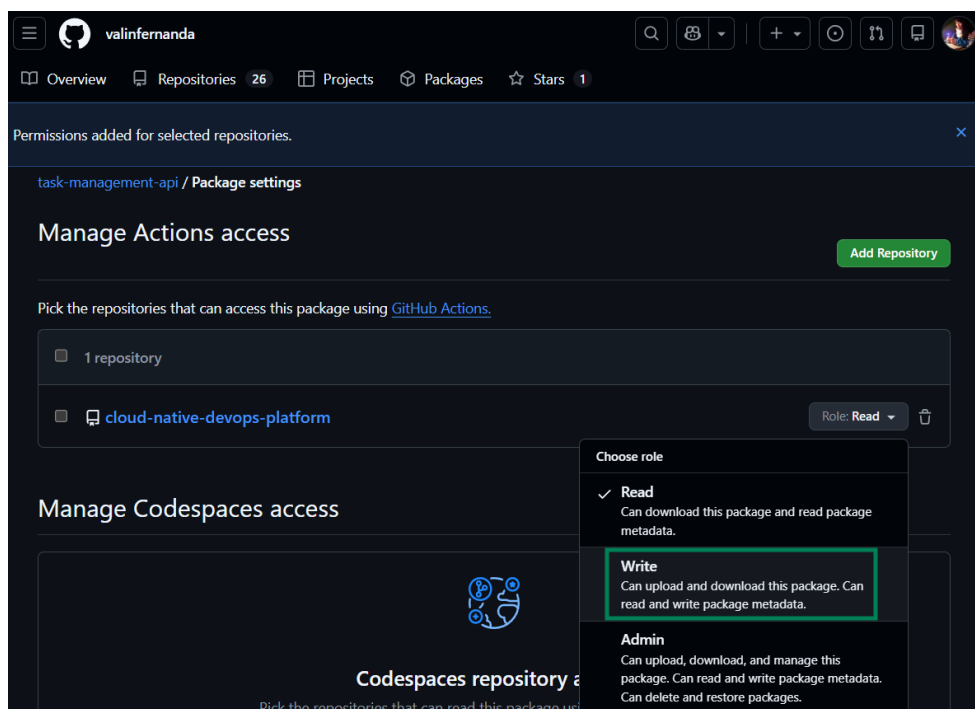
Make sure that the read and write permissions is selected.



The error remains the same. Let's try the other step.

GitHub → Profile → Packages → task-management-api → Package settings

And give 'Write' permission



Still Error. Now add write permission to the packages in deploy.yml

```
8 permissions:
9   id-token: write
10  contents: read
11  packages: write
```

After editing:

Commit the changes to main

GitHub Actions will run automatically again

Check the **Push Docker image** step.

And... the error has now shifted to the Azure Login step. This actually means the GHCR stage has likely been completed successfully. Yeas!

The screenshot shows the GitHub Actions interface for a workflow named 'Add write permission to packages #2'. The workflow is in a failed state, indicated by a red 'x' icon. The left sidebar shows the workflow file and a summary of the run. The main area displays the 'build-and-deploy' job, which failed 6 minutes ago. The job steps are listed, and the 'Azure Login' step is highlighted as failed. The error message for the 'Azure Login' step is: 'Error: Login failed with Error: Using auth-type: SERVICE_PRINCIPAL. Not all values are present. Ensure 'client-id' and 'tenant-id' are supplied.. Double check if the 'auth-type' is correct. Refer to https://github.com/Azure/login#readme for more information.'

Build and Deploy

✖ Add write permission to packages #2

Summary

All jobs

build-and-deploy

Run details

Usage

Workflow file

Annotations

1 error, 1 warning, and 1 notice

build-and-deploy

failed 6 minutes ago in 27s

Search logs

- Set up job 2s
- Checkout repository 1s
- Login to GHCR 0s
- Build Docker image 11s
- Push Docker image 7s
- ✖ Azure Login 0s

1 ▶ Run azure/login@v2

12 Error: Login failed with Error: Using auth-type: SERVICE_PRINCIPAL. Not all values are present. Ensure 'client-id' and 'tenant-id' are supplied.. Double check if the 'auth-type' is correct. Refer to <https://github.com/Azure/login#readme> for more information.

Error: Login failed with Error: Using auth-type: SERVICE_PRINCIPAL. Not all values are present. Ensure 'client-id' and 'tenant-id' are supplied.. Double check if the 'auth-type' is correct. Refer to <https://github.com/Azure/login#readme> for more information.

It means:

GitHub Actions is trying to log in to Azure, but Azure is not receiving the client-id or tenant-id.

How to fix:

Go to your repository → Settings → Secrets and variables → Actions

Then:

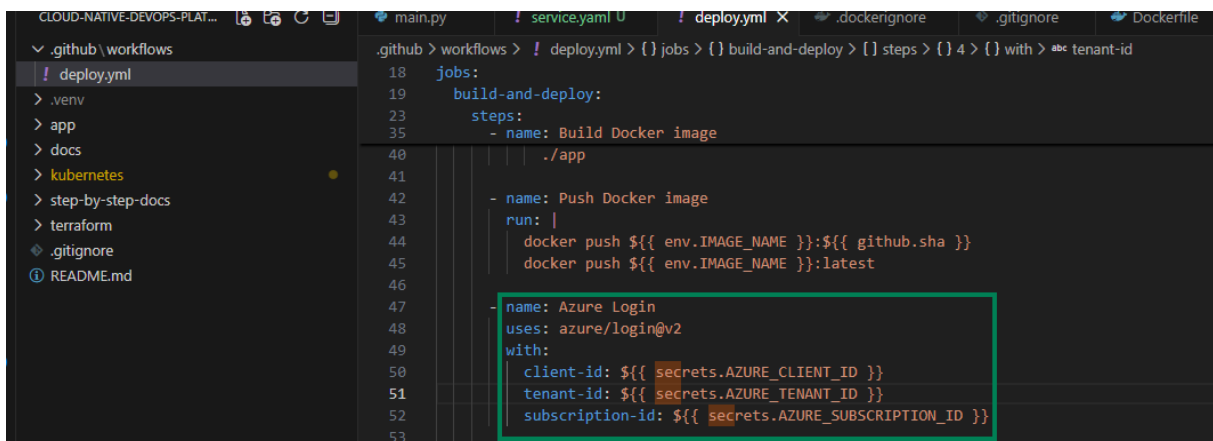
Check the "Repository secrets" section.

Make sure these three secrets exist:

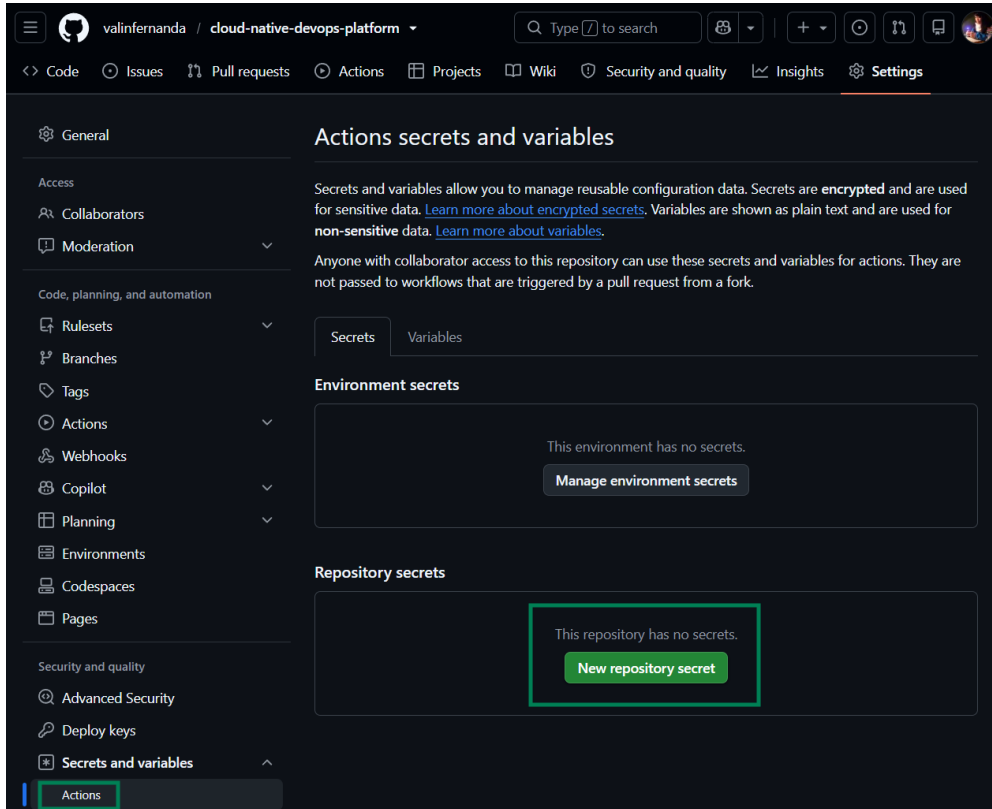
AZURE_CLIENT_ID

AZURE_TENANT_ID

AZURE_SUBSCRIPTION_ID



```
18 jobs:
19   build-and-deploy:
20     steps:
21       - name: Build Docker image
22         run: |
23           docker build -t ${{ env.IMAGE_NAME }}:latest .
24           docker push ${{ env.IMAGE_NAME }}:latest
25
26       - name: Azure Login
27         uses: azure/login@v2
28         with:
29           client-id: ${{ secrets.AZURE_CLIENT_ID }}
30           tenant-id: ${{ secrets.AZURE_TENANT_ID }}
31           subscription-id: ${{ secrets.AZURE_SUBSCRIPTION_ID }}
```



We found the problem. **The three secrets haven't been created yet**, so GitHub Actions doesn't have the information it needs to log in to Azure.

a. Create AZURE_CLIENT_ID

In Azure/PowerShell, run:

```
az identity show `
```

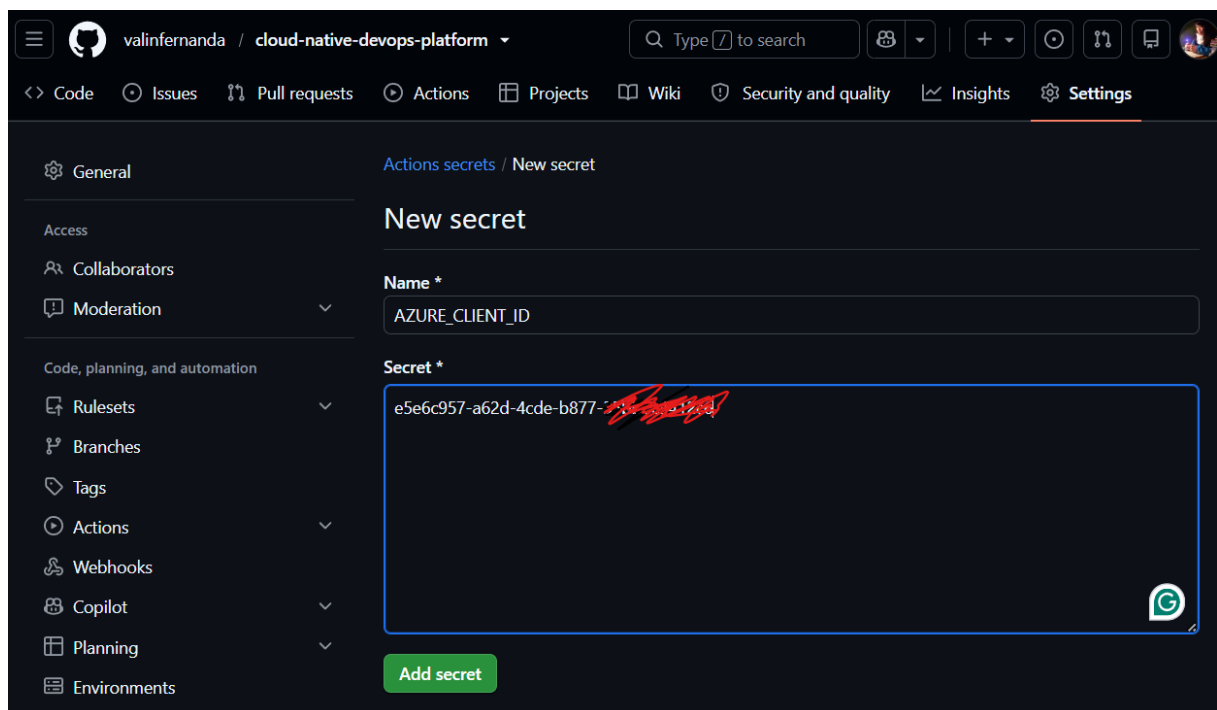
```
--name github-actions-aks `
```

```
--resource-group rg-devops-platform `
```

```
--query clientId `
```

```
-o tsv
```

This will return the **Client ID** of your github-actions-aks Managed Identity.



b. Create AZURE_TENANT_ID

```
az account show --query tenantId -o tsv
```

c. AZURE_SUBSCRIPTION_ID

```
az account show --query id -o tsv
```

At the end, it should be like this:

Repository secret added.

Actions secrets and variables

Secrets and variables allow you to manage reusable configuration data. Secrets are **encrypted** and are used for sensitive data. [Learn more about encrypted secrets](#). Variables are shown as plain text and are used for **non-sensitive** data. [Learn more about variables](#).

Anyone with collaborator access to this repository can use these secrets and variables for actions. They are not passed to workflows that are triggered by a pull request from a fork.

Secrets Variables

Environment secrets

This environment has no secrets.

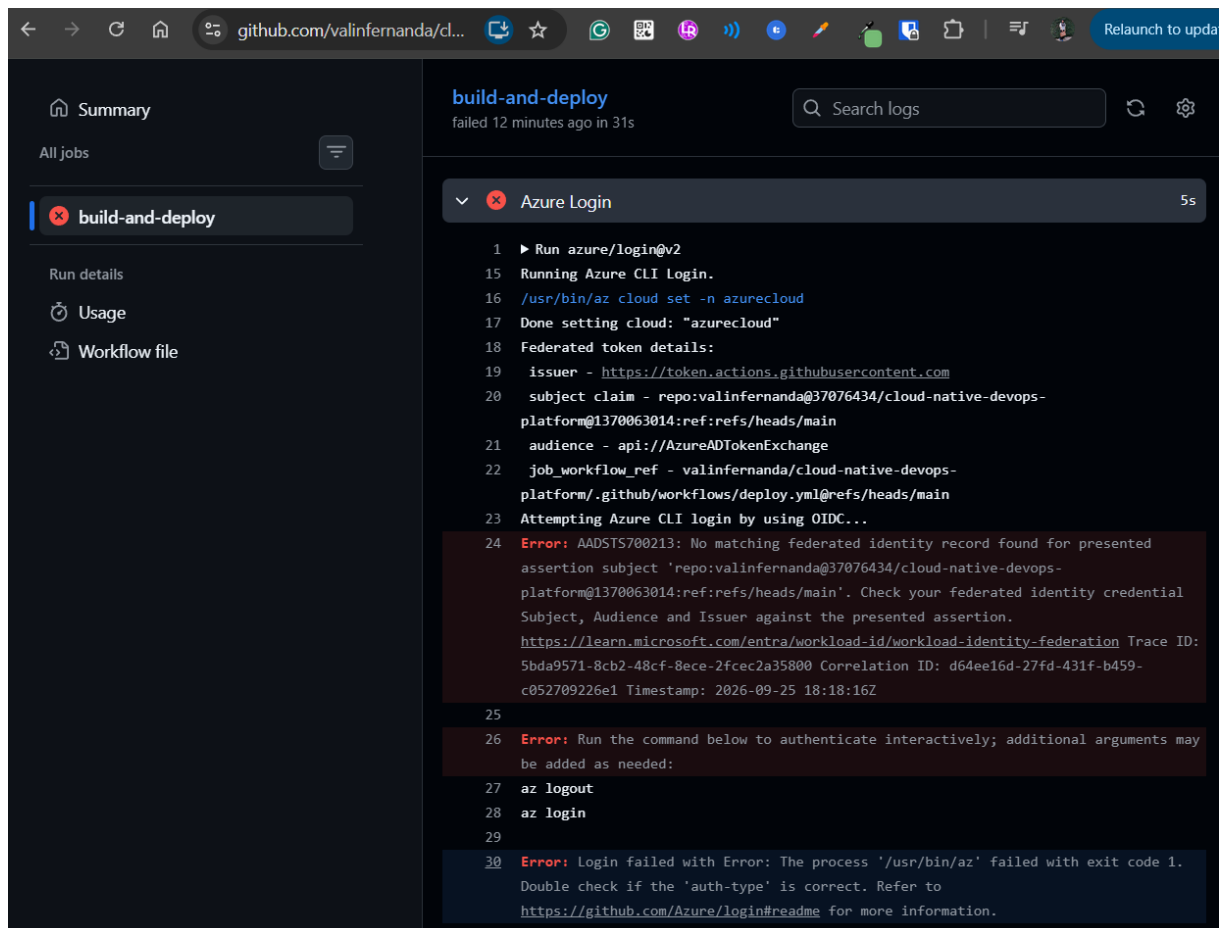
[Manage environment secrets](#)

Repository secrets

[New repository secret](#)

Name	Last updated
 AZURE_CLIENT_ID	23 minutes ago  
 AZURE_SUBSCRIPTION_ID	20 minutes ago  
 AZURE_TENANT_ID	21 minutes ago  

And then.. run the workflow again.



Now, the secrets are correct. The error has changed and is very specific:

AADSTS700213: No matching federated identity record found

This means: Azure received the OIDC token from GitHub, but Azure could not find a matching Federated Credential for that token.

Pay attention to the subject GitHub is sending:

repo:valinfernanda@37076434/cloud-native-devops-
platform@1370063014:ref:refs/heads/main

So, we need to fix the Federated Credential on the github-actions-aks Managed Identity so that it matches the GitHub OIDC token.

How to fix :

a. Check Federated Credential

```
az identity federated-credential list `
```

```
--identity-name github-actions-aks `
```

```
--resource-group rg-devops-platform `
```

```
-o table
```

```

(.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform> az identity federated-credential list `
>> --identity-name github-actions-aks `
>> --resource-group rg-devops-platform `
>> -o table
Issuer                                     Name      ResourceGroup    Subject
-----
https://token.actions.githubusercontent.com  github-actions  rg-devops-platform  repo:valinfernanda/cloud-native-devops-platform:ref:refs/heads/main

```

We found the problem.

Your current **Federated Credential** has:

Subject: repo:valinfernanda/cloud-native-devops-platform:ref:refs/heads/main

But GitHub Actions is currently sending:

Subject: repo:valinfernanda@37076434/cloud-native-devops-platform@1370063014:ref:refs/heads/main

So Azure says: Subject does not match

That's why you get: AADSTS700213: No matching federated identity record found

Why are they different?

Your GitHub Actions workflow is currently using a **repository identifier that includes IDs**, rather than only the repository name.

We need to create a Federated Credential that **matches the subject GitHub is actually sending**.

But don't delete the old one yet

Create a new credential:

```

az identity federated-credential create `
--name github-actions-main `
--identity-name github-actions-aks `
--resource-group rg-devops-platform `
--issuer https://token.actions.githubusercontent.com `
--subject "repo:valinfernanda@37076434/cloud-native-devops-
platform@1370063014:ref:refs/heads/main" `
--audiences api://AzureADTokenExchange

```

Then check:

```

az identity federated-credential list `
--identity-name github-actions-aks `

```

```
--resource-group rg-devops-platform `
```

```
-o table
```

You should see **two credentials**:

github-actions

github-actions-main

```
(.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform> az identity federated-credential list `
>> --identity-name github-actions-aks `
>> --resource-group rg-devops-platform `
>> -o table
Issuer                                     Name           ResourceGroup  Subject
-----
https://token.actions.githubusercontent.com github-actions  rg-devops-platform  repo:valinfernanda/cloud-native-devops-platform:ref:refs/heads/main
https://token.actions.githubusercontent.com github-actions-main rg-devops-platform  repo:valinfernanda@37076434/cloud-native-devops-platform@1370063014:ref:refs/heads/main
```

Then run GitHub Actions again.

Why are we using that long subject?

Because we're **not guessing the subject**. GitHub provided it in the workflow log:

subject claim -

repo:valinfernanda@37076434/cloud-native-devops-
platform@1370063014:ref:refs/heads/main

So we're creating a Federated Credential that exactly matches the assertion GitHub is sending to Azure.

Summary

All jobs

build-and-deploy

Run details

Usage

Workflow file

build-and-deploy
failed 4 minutes ago in 31s

Search logs

- Set up job 3s
- Checkout repository 1s
- Login to GHCR 1s
- Build Docker image 7s
- Push Docker image 7s
- Azure Login 3s
- Set AKS context 2s
- Deploy to AKS 3s

```
1 ▼ Run kubectl apply -f k8s/deployment.yaml
2 kubectl apply -f k8s/deployment.yaml
3 kubectl apply -f k8s/service.yaml
4 shell: /usr/bin/bash -e {0}
5 env:
6   IMAGE_NAME: ghcr.io/valinfernanda/task-management-api
7   RESOURCE_GROUP: rg-devops-platform
8   AKS_CLUSTER: aks-devops-platform
9   AZURE_HTTP_USER_AGENT:
10  AZUREPS_HOST_ENVIRONMENT:
11  KUBECONFIG: /home/runner/work/_temp/kubeconfig_1790361579530
12  KUBE_CONFIG_PATH: /home/runner/work/_temp/kubeconfig_1790361579530
13 error: the path "k8s/deployment.yaml" does not exist
14 Error: Process completed with exit code 1.
```

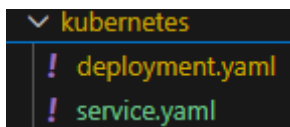
This is a different issue, and it is actually good news: Azure Login and the connection to AKS have already succeeded.

The current error is: error: the path "k8s/deployment.yaml" does not exist

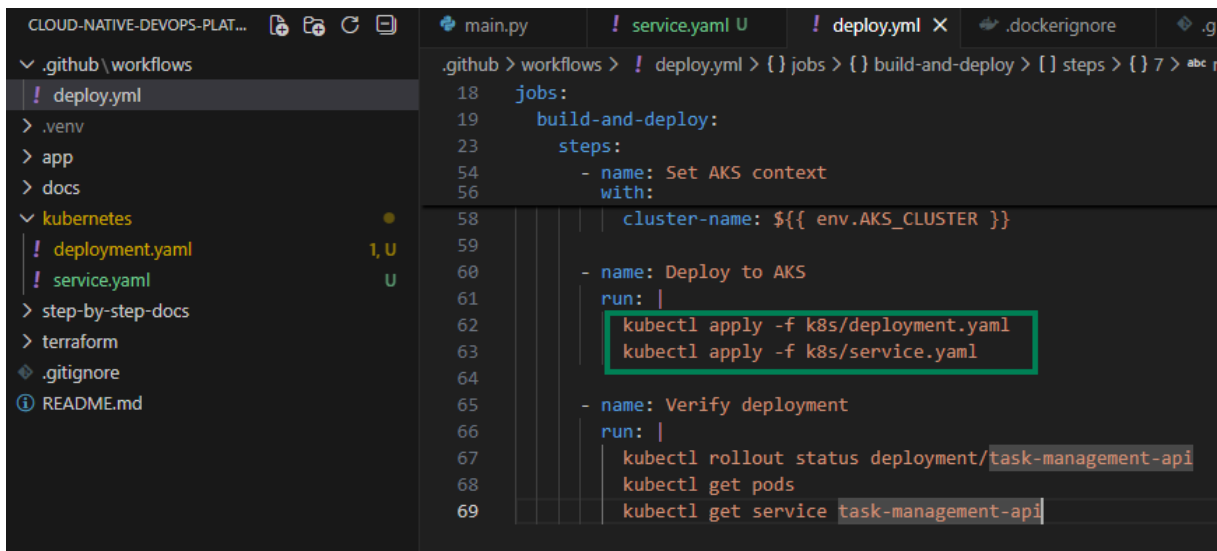
This means GitHub Actions is running: `kubectl apply -f k8s/deployment.yaml`

but in the repository that GitHub Actions checked out, the file does not exist at `k8s/deployment.yaml`.

And... yes, exactly. The problem was simply the **folder name**. We use **'kubernetes'** instead of **'k8s'**



Simply change this:



Save the changes, then run:

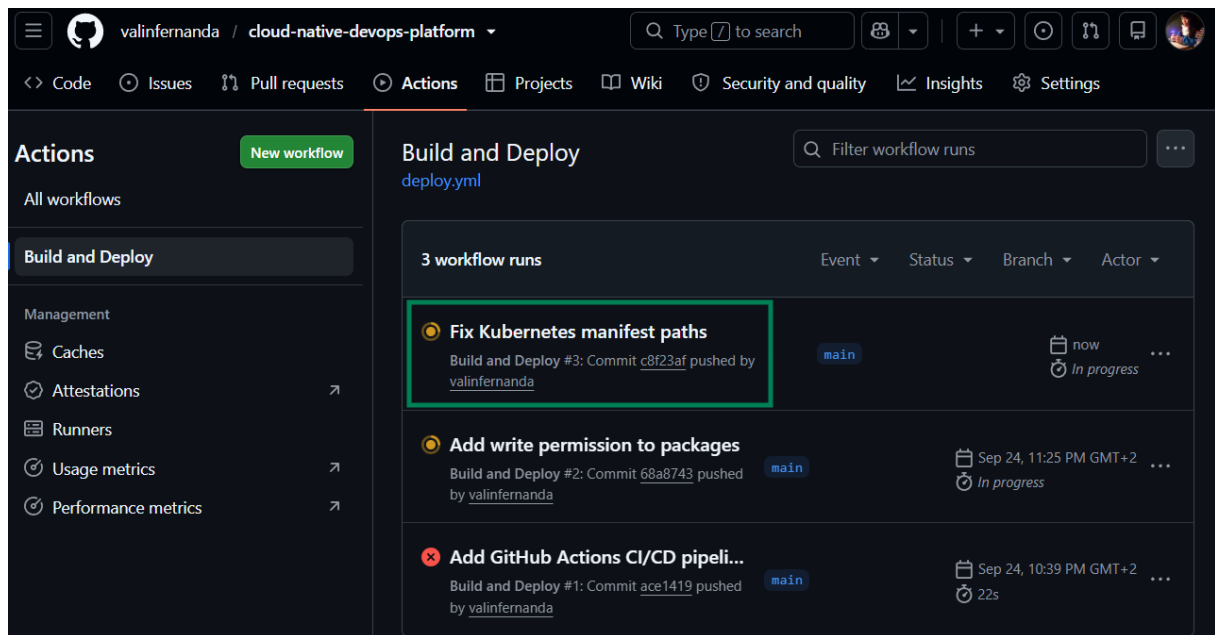
```
git status
```

```
git add .
```

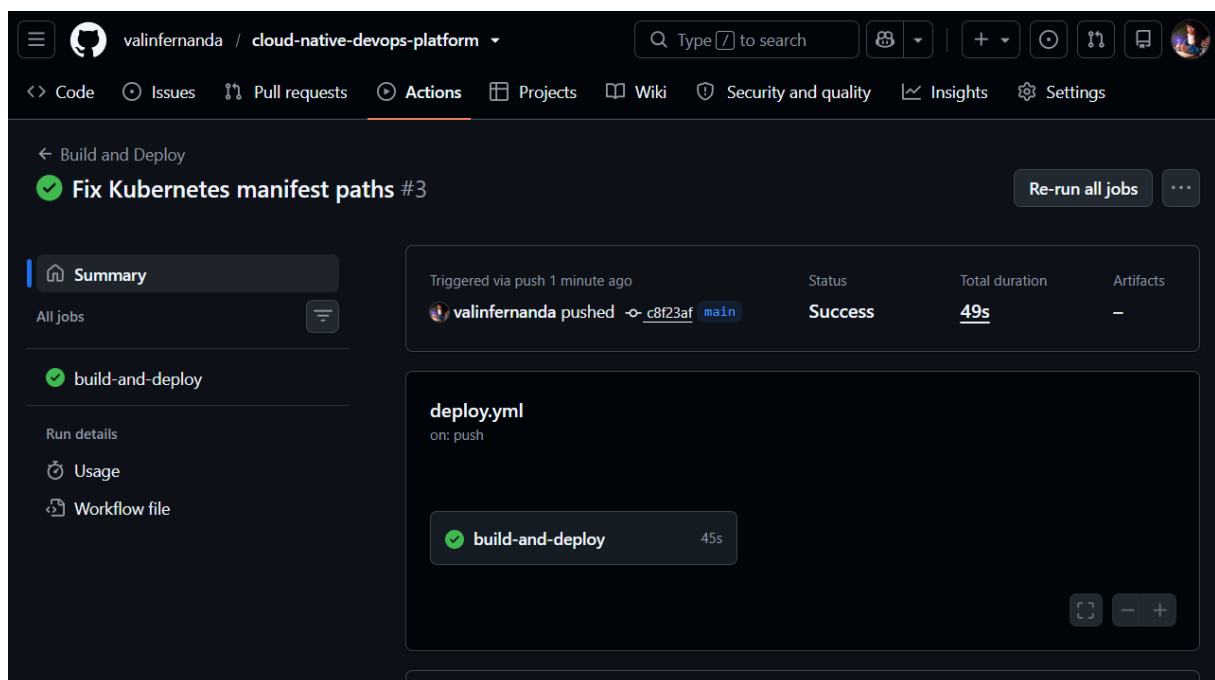
```
git commit -m "Fix Kubernetes manifest paths"
```

```
git push
```

The push will trigger GitHub Actions again.



And finally...



So far we have done:

1. Checkout repository
2. Login to GHCR
3. Build Docker image
4. Push Docker image
5. Azure Login
6. Set AKS context

7. Deploy to AKS

8. Verify deployment

One important thing to validate first

We should verify that the deployment is actually using the new image from the pipeline, rather than just confirming that kubectl apply succeeded.

1. Check the image currently used by the Pods

```
kubectl get pods -l app=task-management-api`
```

```
-o custom-
```

```
columns="POD:.metadata.name,IMAGE:.spec.containers[0].image,STATUS:.status.phase"
```

```
• (.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform> kubectl get pods -l app=task-management-api`  
>> -o custom-columns="POD:.metadata.name,IMAGE:.spec.containers[0].image,STATUS:.status.phase"  
POD IMAGE STATUS  
task-management-api-5965bc845f-2jhcm ghcr.io/valinfernanda/task-management-api:latest Running  
task-management-api-5965bc845f-frlc7 ghcr.io/valinfernanda/task-management-api:latest Running
```

2. Check the rollout

```
kubectl rollout status deployment/task-management-api
```

```
• (.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform> kubectl rollout status deployment/task-management-api  
deployment "task-management-api" successfully rolled out
```

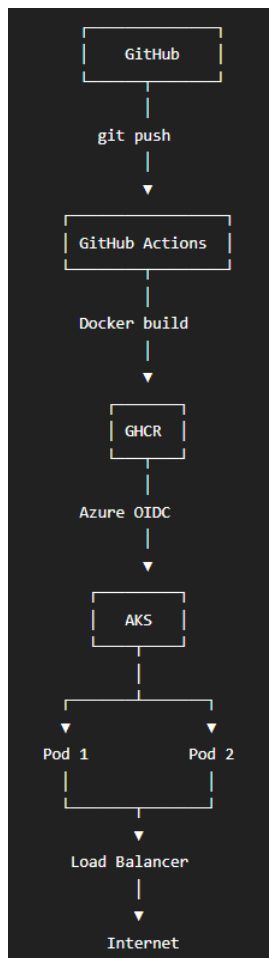
3. Check the Service

```
kubectl get service task-management-api
```

Make sure the EXTERNAL-IP is still present.

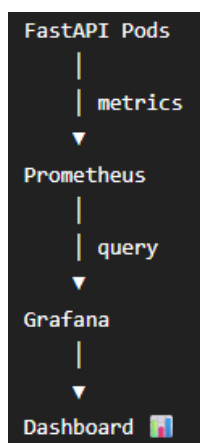
```
• (.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform> kubectl get service task-management-api  
NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S) AGE  
task-management-api LoadBalancer 10.2.154.242 172.169.163.204 80:31887/TCP 31h
```

This confirms that the Pods, deployment, and Service are all running correctly after the pipeline deployment.



25. Monitoring

The monitoring architecture will look like this:



Let us use **kube-prometheus-stack** through **Helm**.

Helm: package manager for Kubernetes

The stack provides:

- Prometheus → collects and stores metrics

- Grafana → visualizes metrics in dashboards
- Alertmanager → handles alerts
- Kubernetes monitoring components

25a. Install Helm

`choco install kubernetes-helm`

```
PS C:\WINDOWS\system32> helm version
version.BuildInfo{Version:"v4.1.4", GitCommit:"05fa37973dc9e42b76e1d2883494c87174b6074f", GitTreeState:"clean", GoVersion:"go1.25.9", KubeClientVersion:"v1.35"}
```

25b. Check cluster

Make sure the nodes and pods are running

`kubectl get nodes`

`kubectl get pods`

```
• (.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform> kubectl get nodes
NAME                                STATUS    ROLES    AGE    VERSION
aks-system-32981788-vmss000000     Ready    <none>   4d6h   v1.35.7
• (.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform> kubectl get pods
NAME                                READY    STATUS    RESTARTS   AGE
task-management-api-5965bc845f-2jhcm 1/1      Running   0           44h
task-management-api-5965bc845f-fr1c7 1/1      Running   0           44h
```

25c. Create the Monitoring Namespace

`kubectl create namespace monitoring`

This creates a separate Kubernetes namespace called monitoring, where we can deploy monitoring tools such as Prometheus and Grafana.

In Kubernetes, a **namespace** is like a separate workspace or folder inside your cluster. A namespace is **not a server or a new cluster**. A namespace is simply a **logical separation within a single Kubernetes cluster**.

`kubectl get namespaces`

```
• (.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform> kubectl get namespaces
NAME              STATUS    AGE
default           Active    4d6h
kube-node-lease   Active    4d6h
kube-public       Active    4d6h
kube-system       Active    4d6h
monitoring        Active    83s
```

26. Add Helm repository

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
```

then

```
helm repo update
```

```
• (.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform> helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
"prometheus-community" has been added to your repositories
• (.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform> helm repo update
Hang tight while we grab the latest from your chart repositories...
...Successfully got an update from the "prometheus-community" chart repository
Update Complete. ♀Happy Helming!♀
```

Notes: The Helm CLI runs on the machine where you execute the Helm command. However, the applications that Helm installs are created inside the Kubernetes cluster (AKS).

Helm is a client-side tool that uses the Kubernetes API to install and manage applications in a Kubernetes cluster.

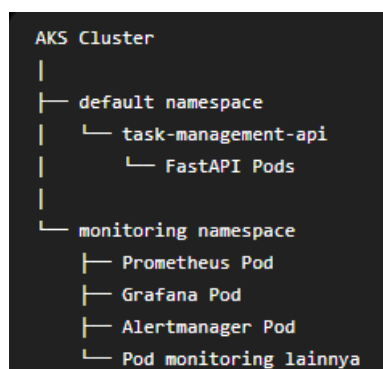
27. Install kube-prometheus-stack

```
helm install monitoring prometheus-community/kube-prometheus-stack`
```

```
--namespace monitoring
```

Means: Helm, install the kube-prometheus-stack chart from the prometheus-community repository, give this installation the release name monitoring, and put its Kubernetes resources in the monitoring namespace

Concept:



- Helm = a tool used to install applications into Kubernetes.
- kube-prometheus-stack = the monitoring stack being installed.
- --namespace monitoring = the namespace where those resources will be created.



The monitoring components will create additional pods after monitoring is installed.

When you run: **kubectl get pods**

you will only see pods in the **current/default namespace**.

To see the pods in the **monitoring namespace**, use: **kubectl get pods -n monitoring**

The **-n monitoring** option tells kubectl to look specifically in the monitoring namespace.

28. Check the Monitoring Pods We Just Created

kubectl get pods -n monitoring

```

(.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform> kubectl get pods -n monitoring
NAME                                                    READY   STATUS    RESTARTS   AGE
alertmanager-monitoring-kube-prometheus-alertmanager-0 2/2     Running   0           11m
monitoring-grafana-77598c4d49-km9mq                    3/3     Running   0           12m
monitoring-kube-prometheus-operator-895b6bb59-6dsmn    1/1     Running   0           12m
monitoring-kube-state-metrics-6d8ffd8867-85jxw         1/1     Running   0           12m
monitoring-prometheus-node-exporter-dtj6b              1/1     Running   0           12m
prometheus-monitoring-kube-prometheus-prometheus-0     2/2     Running   0           11m

```

check the services in the monitoring namespace

kubectl get svc -n monitoring

```

(.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform> kubectl get svc -n monitoring
NAME                                TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)                                AGE
alertmanager-operated              ClusterIP    None         <none>        9093/TCP,9094/TCP,9094/UDP            24m
monitoring-grafana                 ClusterIP    10.2.98.230  <none>        80/TCP                                24m
monitoring-kube-prometheus-alertmanager ClusterIP    10.2.58.254  <none>        9093/TCP,8080/TCP                    24m
monitoring-kube-prometheus-operator ClusterIP    10.2.86.99   <none>        443/TCP                                             24m
monitoring-kube-prometheus-prometheus ClusterIP    10.2.108.97  <none>        9090/TCP,8080/TCP                    24m
monitoring-kube-state-metrics       ClusterIP    10.2.18.67   <none>        8080/TCP                                           24m
monitoring-prometheus-node-exporter ClusterIP    10.2.26.26   <none>        9100/TCP                                           24m
prometheus-operated                 ClusterIP    None         <none>        9090/TCP                                         24m

```

Check Helm status release

helm status monitoring -n monitoring

helm status [RELEASE_NAME] -n [NAMESPACE]

↑ ↑
 monitoring monitoring

- **First monitoring** → the **Helm release name**
- **Second monitoring** → the **Kubernetes namespace**

They just happen to have the same name.

For example, you could have:

```
helm install prometheus prometheus-community/kube-prometheus-stack \
  --namespace monitoring
```

Then you would check it with:

```
helm status prometheus -n monitoring
```

So:

Release name = the name of the Helm installation.

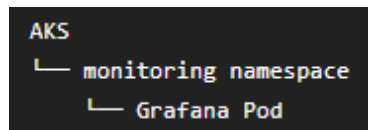
Namespace = the Kubernetes area where the resources are installed

29. Access Grafana

Don't expose Grafana to the internet. We'll use port-forwarding instead:

Why shouldn't we expose Grafana to the internet?

Grafana is currently running inside AKS:



```
kubectrl port-forward -n monitoring svc/monitoring-grafana 3000:80
```

```

(.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform> kubectrl port-forward -n monitoring svc/monitoring-grafana 3000:80
Forwarding from 127.0.0.1:3000 -> 3000
Forwarding from [::1]:3000 -> 3000
  
```

For development, **port-forward allows you to access Grafana without making it publicly accessible.**

kubectrl port-forward -- Create a connection from a port on my computer to a resource in Kubernetes.

-n monitoring ---"Look for the resource in the monitoring namespace."

Because our Grafana is in:

svc/monitoring-grafana

svc = **Service**.

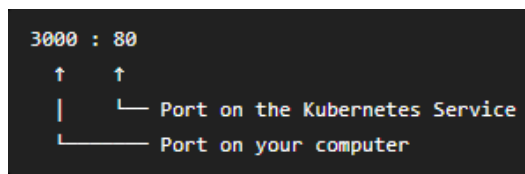
monitoring-grafana = the name of the Grafana Service.

So we are saying: "Forward the connection to the Grafana Service."

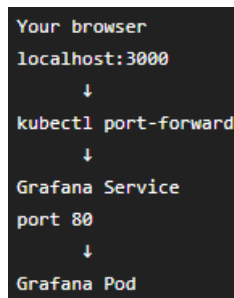
It is **not being exposed to the internet**.

3000:80

This is the most important part:



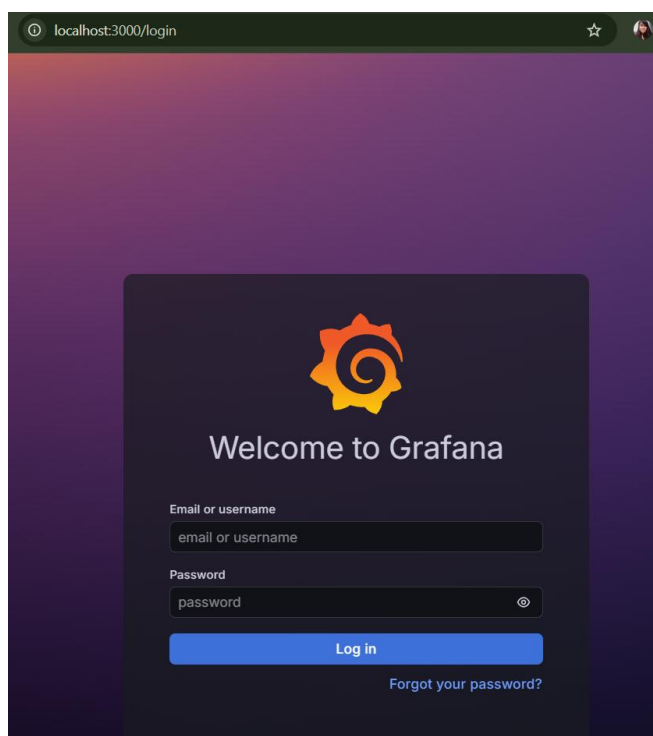
So the connection looks like:



Then you can open: <http://localhost:3000> in your browser to access Grafana.

Then open your browser and go to:

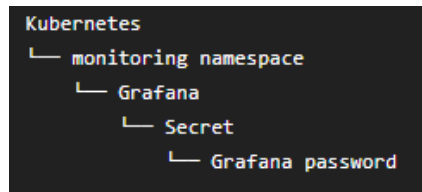
<http://localhost:3000>



To log in, we need to **retrieve the Grafana password from a Kubernetes Secret**.

The password is stored in a Kubernetes Secret rather than directly in the Pod configuration.

Conceptually:



29a. Get the Grafana username

```
kubectl get secret -n monitoring monitoring-grafana `
-o jsonpath="{.data.admin-user}" | ForEach-Object {
[System.Text.Encoding]::UTF8.GetString([System.Convert]::FromBase64String($_))
}
```

```
PS C:\Users\Valin\Downloads\cloud-native-devops-platform> kubectl get secret -n monitoring monitoring-grafana `
>> -o jsonpath="{.data.admin-user}" | ForEach-Object { [System.Text.Encoding]::UTF8.GetString([System.Convert]::FromBase64String($_)) }
admin
```

29b. Get the Grafana password

```
kubectl get secret -n monitoring monitoring-grafana `
-o jsonpath="{.data.admin-password}" | ForEach-Object {
[System.Text.Encoding]::UTF8.GetString([System.Convert]::FromBase64String($_))
}
```

```
PS C:\Users\Valin\Downloads\cloud-native-devops-platform> kubectl get secret -n monitoring monitoring-grafana `
>> -o jsonpath="{.data.admin-password}" | ForEach-Object { [System.Text.Encoding]::UTF8.GetString([System.Convert]::FromBase64String($_)) }
wY18wthQX1Rx61d9LwZNjGYY6XO5O4vXqOEtr8mv
```

wY18wthQX1Rx61d9LwZNjGYY6XO5O4vXqOEtr8mv

Okay, that means **Grafana is already running on AKS**.

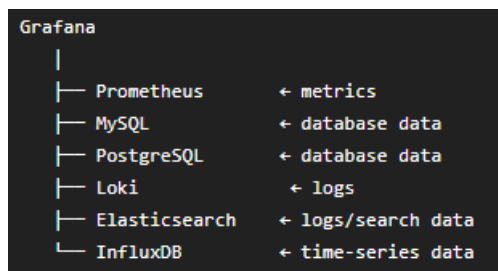
Now, let's not jump straight into creating dashboards manually. First, we need to make sure **Grafana is actually receiving data from Prometheus**.

Prometheus = collects the data.

Grafana = displays the data.

Notes: Grafana can use many different data sources, not only Prometheus.

For example:



So why are we using Prometheus?

Because our goal is **Kubernetes monitoring**.

Prometheus is designed to collect **metrics**, such as:

CPU usage

Memory usage

Pod count

Request rate

Network traffic

The Grafana installed by kube-prometheus-stack is normally already connected to the Prometheus installed by the same stack.

Next step ...

30. Check the Prometheus datasource

In Grafana:

1. Go to:

☰ → **Connections** → **Data sources**

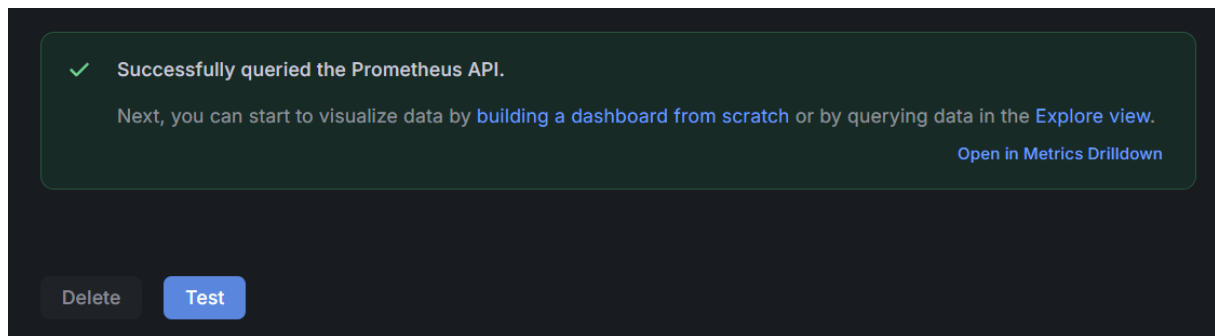
Look for: **Prometheus**

Since we installed kube-prometheus-stack, Prometheus should normally already be configured as a datasource.

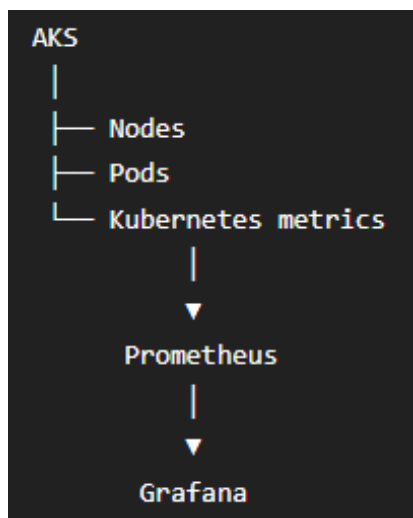
2. Click Prometheus

Then find: **Save & test**

If you see: **Data source is working**



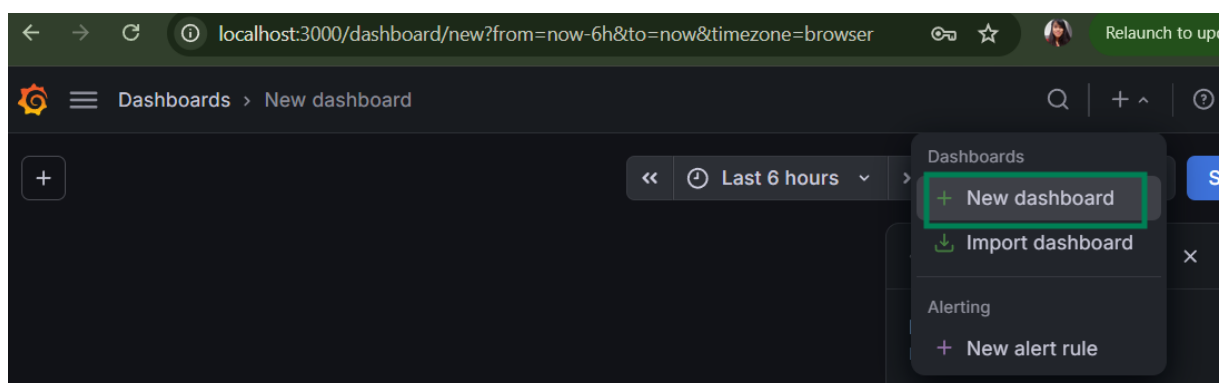
then we know the monitoring pipeline is working:



31 . Building useful portfolio dashboards

In Grafana, click:

Dashboards → New → New dashboard

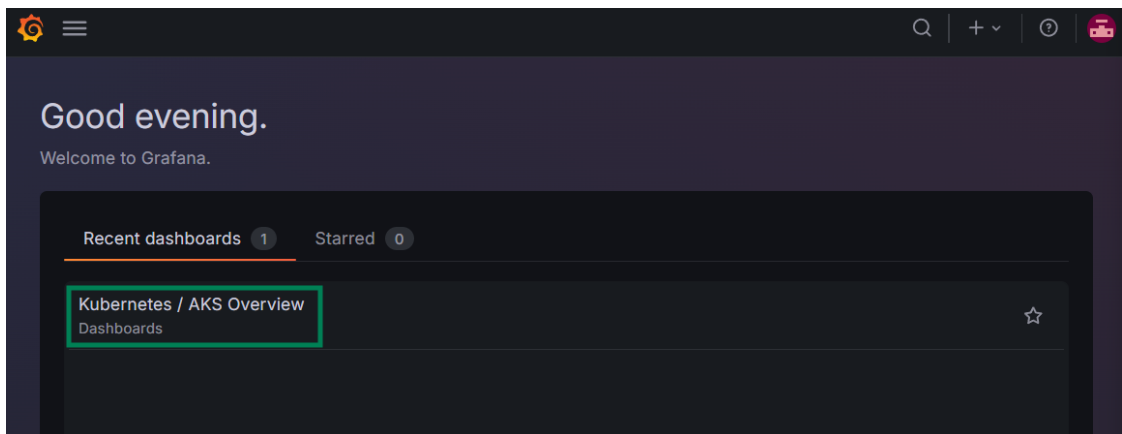


Then choose: **Add visualization**

31a. First Metric: CPU Usage

`100 - (avg by (instance) (rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100)`

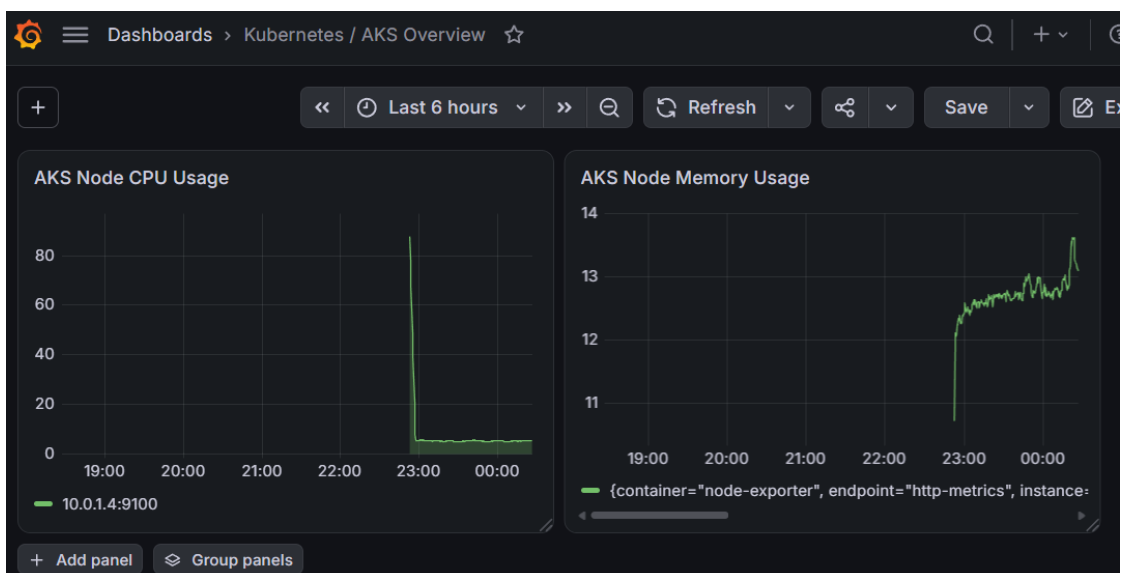
You should see a graph/table with your AKS node(s).



31b. Memory Metric

Give this title: AKS Node Memory Usage

```
100 * (1 - (
node_memory_MemAvailable_bytes
/
node_memory_MemTotal_bytes
))
```



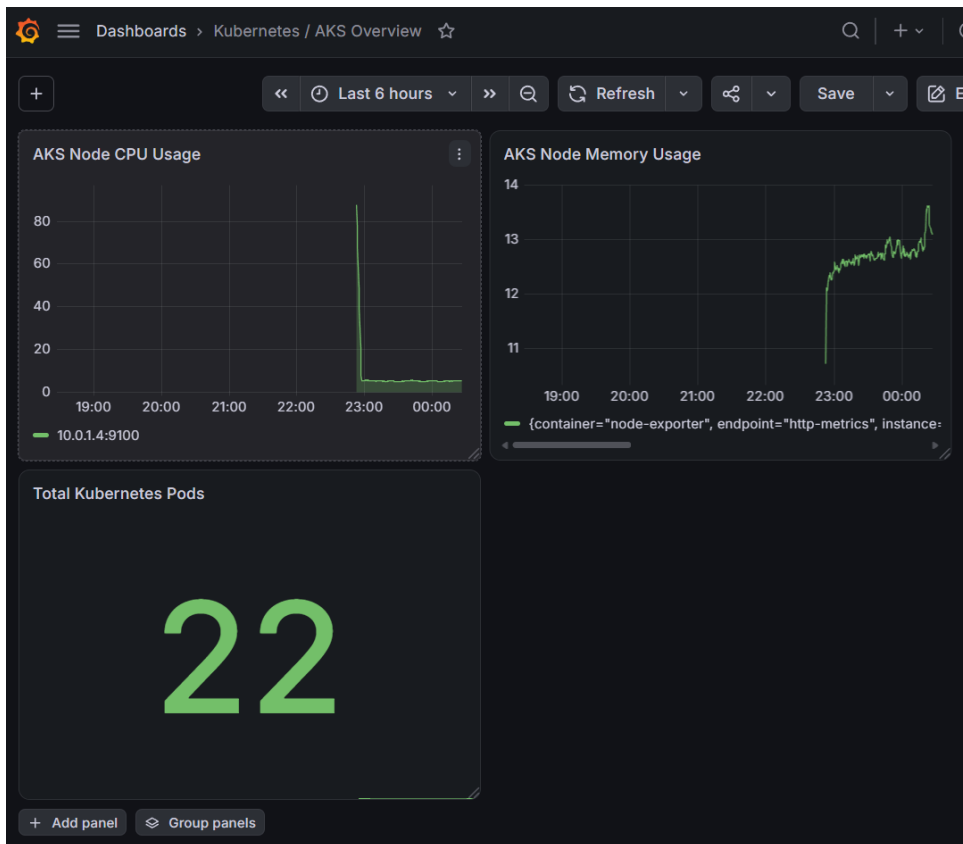
31c. Add Pod Count Metric

```
count(kube_pod_info)
```

Set the visualization to: **Stat**

A Stat panel is better here than a graph because we mainly want to see the **current number**.

Set the title to: **Total Kubernetes Pods**

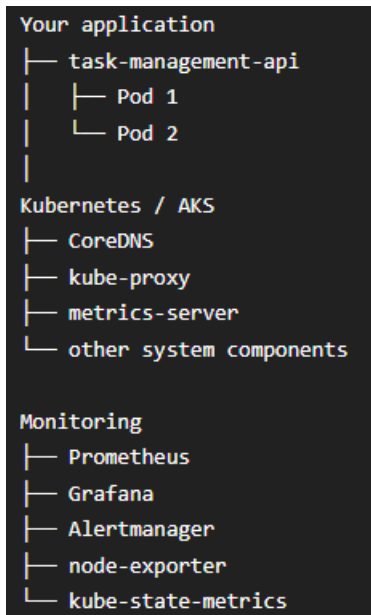


22 Pods can be completely normal. Remember, the query:

```
count(kube_pod_info)
```

counts **all Pods visible to Prometheus across the cluster**, not just your task-management-api.

Your cluster has things like:



So after installing **kube-prometheus-stack**, the Pod count increased because the monitoring stack itself runs several Pods.

Let's verify exactly what your 22 Pods are

In PowerShell, run: **kubectl get pods -A**

-A means : show Pods from all namespaces

```
PS C:\Users\Valin\Downloads\cloud-native-devops-platform> kubectl get pods -A
```

NAMESPACE	NAME	READY	STATUS	RESTARTS	AGE
default	task-management-api-5965bc845f-2jhcm	1/1	Running	0	47h
default	task-management-api-5965bc845f-frlc7	1/1	Running	0	47h
kube-system	azure-cns-xhxnt	1/1	Running	0	21h
kube-system	azure-ip-masq-agent-fb8ks	1/1	Running	0	4d9h
kube-system	cloud-node-manager-z9nnl	1/1	Running	0	45h
kube-system	coredns-5d474ff6db-72qpx	1/1	Running	0	4d9h
kube-system	coredns-5d474ff6db-7vmxt	1/1	Running	0	4d9h
kube-system	coredns-autoscaler-6769f8f9b-z7jjz	1/1	Running	0	4d9h
kube-system	csi-azuredisk-node-ntlfj	3/3	Running	0	45h
kube-system	csi-azurefile-node-grbdl	4/4	Running	0	45h
kube-system	konnnectivity-agent-6869bcc4b8-c9c55	1/1	Running	0	45h
kube-system	konnnectivity-agent-6869bcc4b8-cd5zj	1/1	Running	0	45h
kube-system	konnnectivity-agent-autoscaler-57c596c6fd-sgl2c	1/1	Running	0	45h
kube-system	kube-proxy-w9fvw	1/1	Running	0	45h
kube-system	metrics-server-7c4d5d8fdb-tjsk6	2/2	Running	0	45h
kube-system	metrics-server-7c4d5d8fdb-zj8l6	2/2	Running	0	45h
monitoring	alertmanager-monitoring-kube-prometheus-alertmanager-0	2/2	Running	0	116m
monitoring	monitoring-grafana-77598c4d49-km9mq	3/3	Running	0	116m
monitoring	monitoring-kube-prometheus-operator-895b6bb59-6dsmn	1/1	Running	0	116m
monitoring	monitoring-kube-state-metrics-6d8ffd8867-85jxw	1/1	Running	0	116m
monitoring	monitoring-prometheus-node-exporter-dtj6b	1/1	Running	0	116m
monitoring	prometheus-monitoring-kube-prometheus-prometheus-0	2/2	Running	0	116m

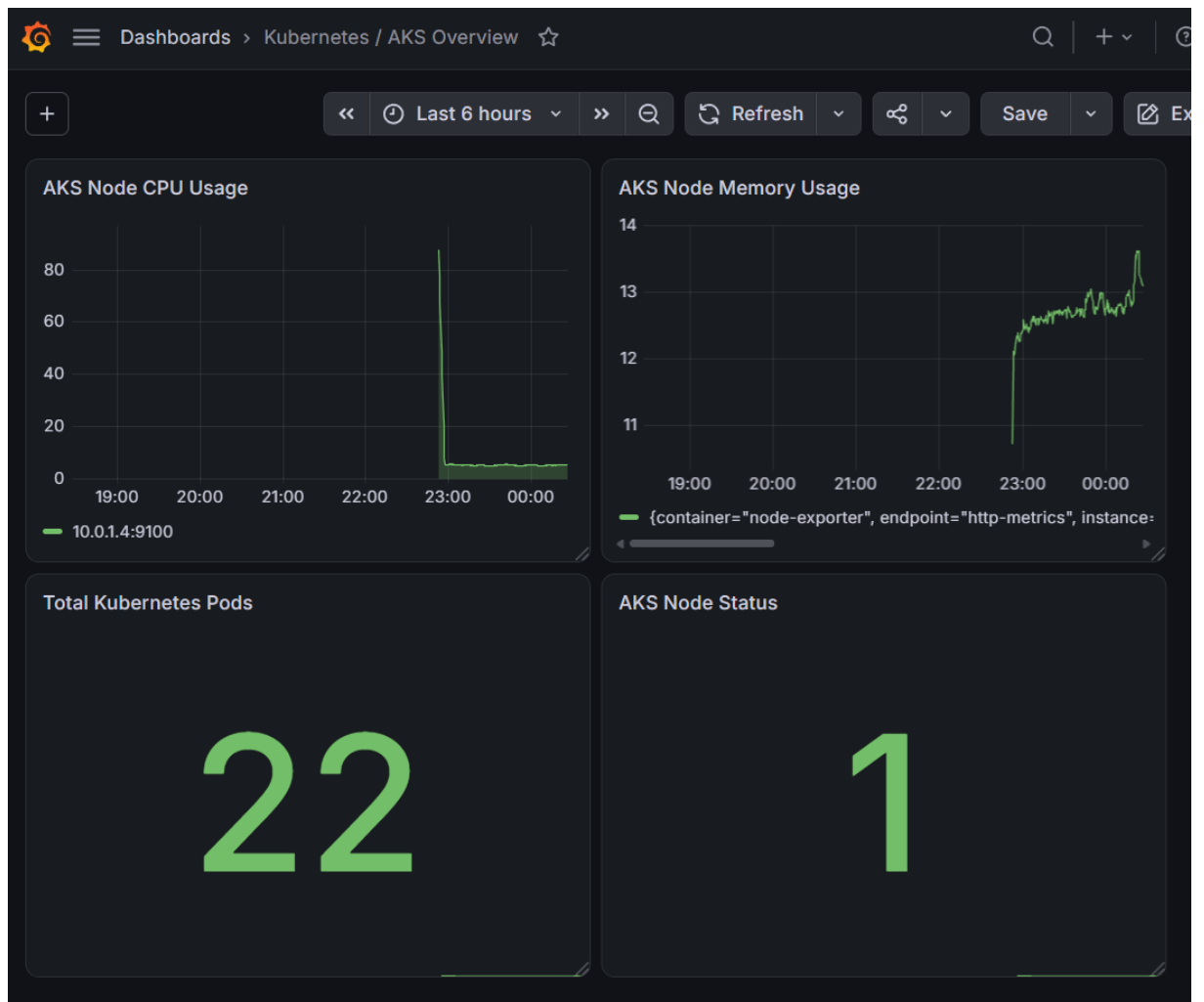
31d. Node Status Metric

kube_node_status_condition{condition="Ready",status="true"}

Set the visualization to: **Stat**

Set the title: **AKS Node Status**

You should get a value of **1** if the node is Ready.



32. Check the Task Management API metrics

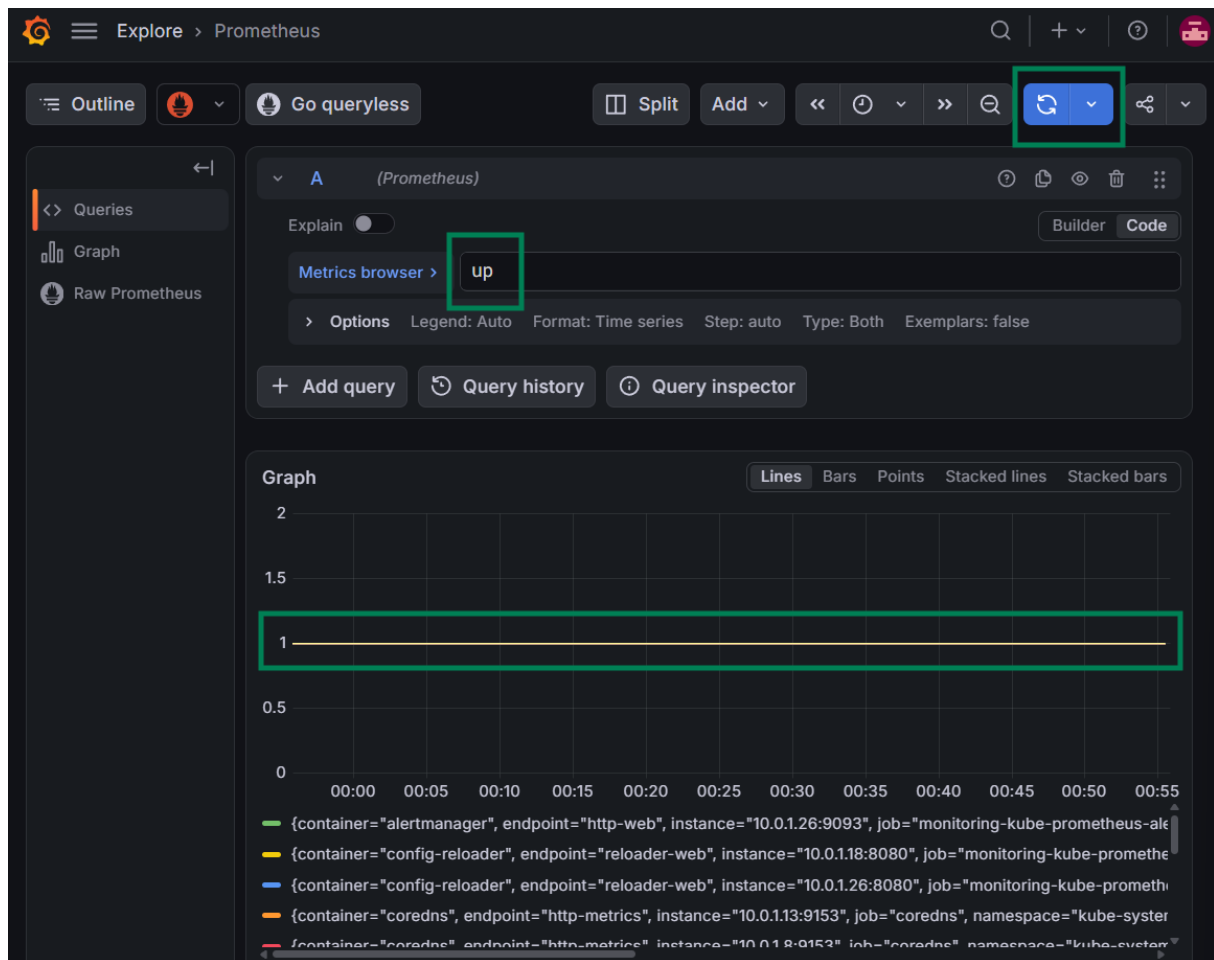
Open **Grafana** → **Explore**.

Make sure the datasource is: **Prometheus**

In the query box, enter: **up**

Then click **Run query**.

You should see several results. 1 means the target is currently reachable/up.



Notes: So far, we made **Kubernetes Cluster dashboard only** and our FastAPI application **doesn't currently expose a Prometheus /metrics endpoint**, so we shouldn't pretend Prometheus is already collecting application-level metrics like request count or latency.

33. Add Prometheus metrics to FastAPI

At the top, we'll eventually add a Prometheus library and middleware.

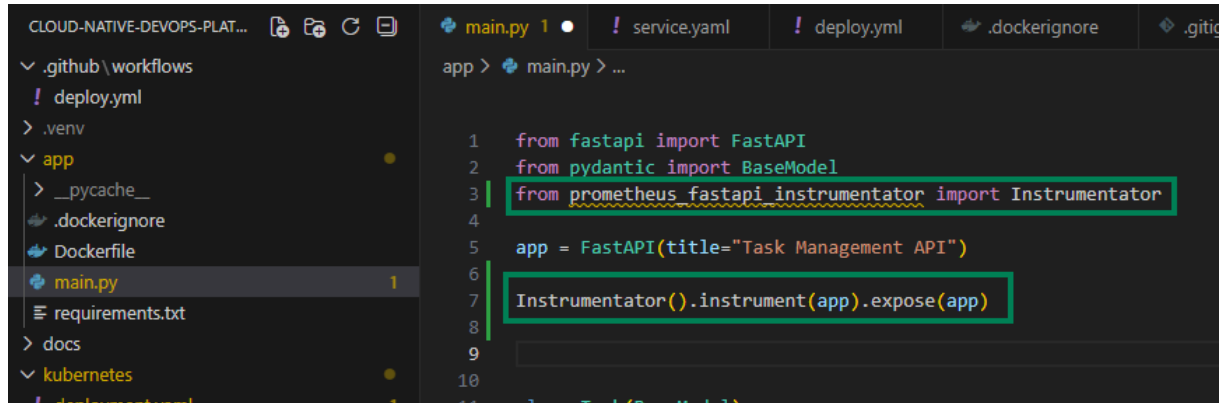
We'll use `prometheus-fastapi-instrumentator`, which automatically exposes metrics such as:

- request count
- request duration/latency
- HTTP status codes
- requests by endpoint

Open: app/main.py

```
from prometheus_fastapi_instrumentator import Instrumentator
```

```
Instrumentator().instrument(app).expose(app)
```



Next, add the Python dependency

Open: app/requirements.txt

and add: **prometheus-fastapi-instrumentator**

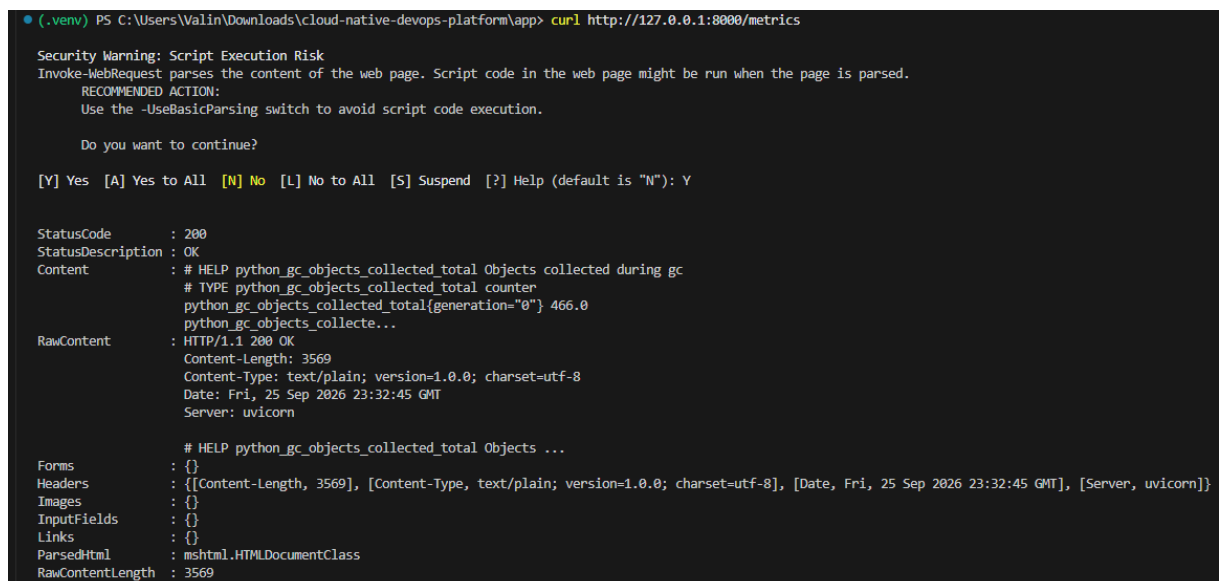
```
pip install prometheus-fastapi-instrumentator
```

34. Test locally

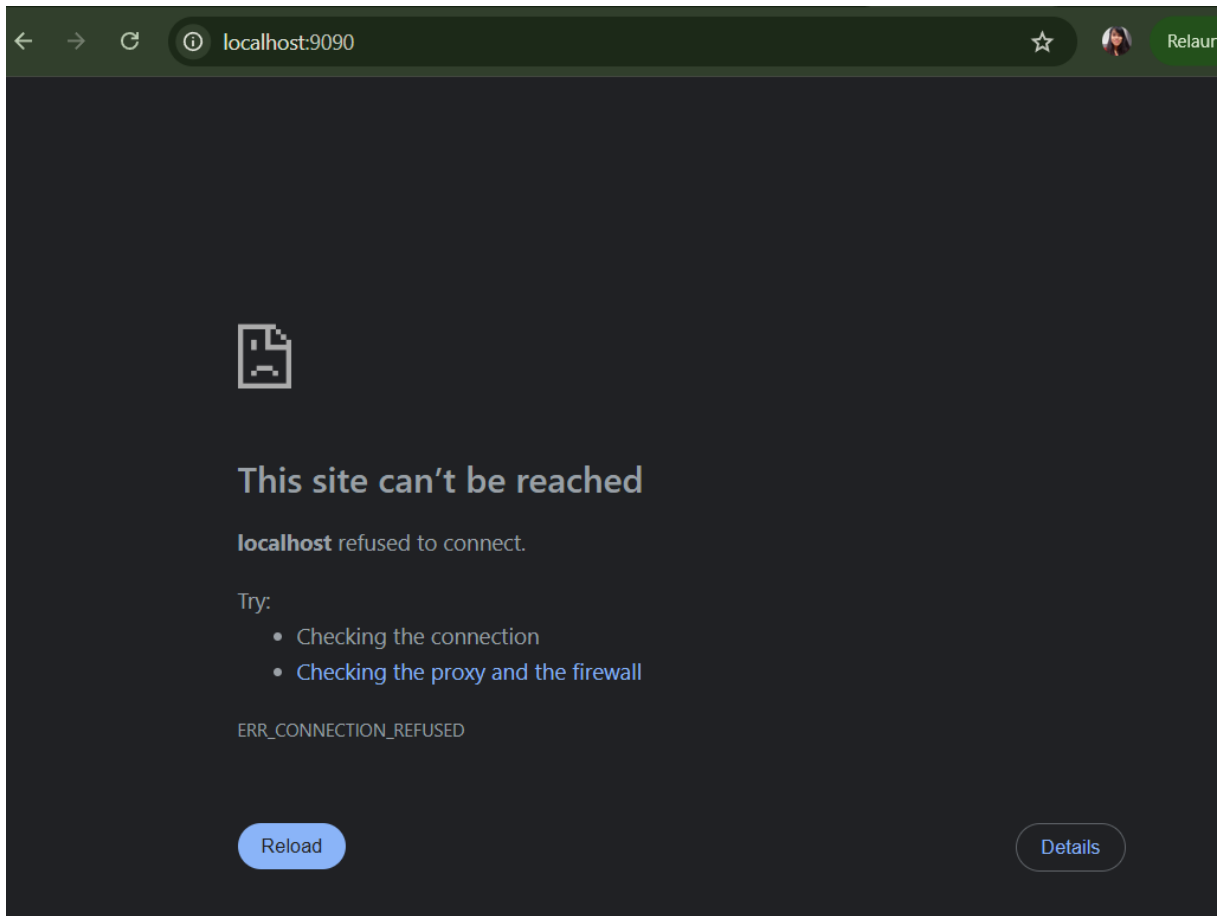
From the project root: **cd app**

```
pip install -r requirements.txt and then: uvicorn main:app --reload
```

Open another PowerShell window and run: **curl http://127.0.0.1:8000/metrics**



Prometheus typically uses port 9090 for its web UI. However, this only applies if Prometheus is actually running with that configuration/default port. So, **don't open localhost:9090 immediately** until we have confirmed that Prometheus is running.



We have confirmed:

- **Local FastAPI:** /metrics → 200 OK ✓
- **AKS Pod:** /metrics → 404 Not Found ✗

So our **local code has /metrics**, but the Docker image currently running in AKS apparently does not.

1. Check which image the AKS Pod is using

```
kubectl get pod task-management-api-5965bc845f-2jhcm -o  
jsonpath="{.spec.containers[0].image}"
```

```
• (.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform\app> kubectl get pod task-management-api-5965bc845f-2jhcm -o jsonpath="{.spec.containers[0].image}"  
ghcr.io/valinfernanda/task-management-api:latest
```

```
kubectl exec task-management-api-5965bc845f-2jhcm -- python -c "import
urllib.request;
print(urllib.request.urlopen('http://127.0.0.1:8000/openapi.json').read().decode()[:5000]
)"
```

Confirmed that the /metrics is not registered and that confirms the application inside the AKS Pod doesn't have that route.

From the app folder, run:

```
(.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform\app> Select-String -Path .\main.py -Pattern "Instrumentator|metrics"

main.py:3:from prometheus_fastapi_instrumentator import Instrumentator
main.py:7:Instrumentator().instrument(app).expose(app)
```

LOCAL

↓

AKS

↓

If the local main.py contains the Instrumentator code but the AKS Pod returns 404, then the **Docker image needs to be rebuilt and pushed to GHCR**, and then AKS needs to use that updated image.

So the next step is to **build a new Docker image containing this updated code**, push it to GHCR, and then let AKS pull the new image.

Solution:

1. Rebuild Docker Image

```
docker build -t ghcr.io/valinfernanda/task-management-api:latest .
```

2. Push image baru

```
docker build -t ghcr.io/valinfernanda/task-management-api:latest .
```

```
● (.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform\app> kubectl rollout restart deployment task-management-api
deployment.apps/task-management-api restarted
○ (.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform\app> kubectl rollout status deployment task-management-api
Waiting for deployment "task-management-api" rollout to finish: 1 old replicas are pending termination...
```

It takes some time..meanwhile, check whether the old Pod is stuck in Terminating

kubectl get pods

```
● PS C:\Users\Valin\Downloads\cloud-native-devops-platform> kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
task-management-api-5965bc845f-2jhcm	1/1	Running	0	2d14h
task-management-api-8d65c7b74-6f2p7	0/1	CrashLoopBackOff	6 (3m13s ago)	9m14s
task-management-api-8d65c7b74-csb2k	0/1	CrashLoopBackOff	6 (3m37s ago)	9m18s

we need to find **why the new image is crashing**.

Run this for one of the new Pods:

```
kubectl logs task-management-api-8d65c7b74-6f2p7
```

If the output is short or doesn't show the actual error, run:

```
kubectl logs task-management-api-8d65c7b74-6f2p7 --previous
```

Why --previous?

The container is repeatedly crashing. --previous asks Kubernetes for the logs from the **previous crashed container instance**, which is usually exactly what we need.

```
● PS C:\Users\Valin\Downloads\cloud-native-devops-platform> kubectl logs task-management-api-8d65c7b74-6f2p7 --previous
exec /usr/local/bin/uvicorn: exec format error
```

Ah, now we have the real cause. The error is:

exec /usr/local/bin/uvicorn: exec format error

This is not a Prometheus problem and not a Kubernetes configuration problem.

It means the Docker image you're running was built for a **different CPU architecture** than the AKS node (**like our case before, see step number 17**)

Our AKS node is likely amd64, while the image you just built on your machine may be arm64 (or vice versa).

a. Confirm the AKS node architecture

```
kubectl get nodes -o wide
```

Then:

```
kubectl get node aks-system-32981788-vmss000000 -o  
jsonpath="{.status.nodeInfo.architecture}"
```

```
PS C:\Users\Valin\Downloads\cloud-native-devops-platform> kubectl get node aks-system-32981788-vmss000000 -o jsonpath="{.status.nodeInfo.architecture}"  
arm64
```

Confirmed it is arm64

2. Check your local Docker architecture

```
docker version --format '{{.Server.Arch}}'
```

```
PS C:\Users\Valin\Downloads\cloud-native-devops-platform> docker version --format '{{.Server.Arch}}'  
amd64
```

And this is amd64. Now we found the problem. Our architectures are opposite:

AKS node → arm64

Docker engine → amd64

That's why the new Pod gives: exec /usr/local/bin/uvicorn: exec format error

Fix: build the image for ARM64

Go back to your app directory:

```
cd C:\Users\Valin\Downloads\cloud-native-devops-platform\app
```

Then build specifically for ARM64:

```
docker build --platform linux/arm64 -t ghcr.io/valinfernanda/task-management-  
api:latest .
```

Wait for it to finish. It can take a while, especially because we are building for:

linux/arm64

while our Docker engine is: amd64

So Docker is doing a cross-platform build, which can be significantly slower.

```
(.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform\app> docker build --platform linux/arm64 -t ghcr.io/valinfernanda/task-management-api:latest .
[+] Building 620.0s (7/9)
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 268B
=> [internal] load metadata for docker.io/library/python:3.12-slim
=> [internal] load .dockerignore
=> => transferring context: 87B
=> [1/5] FROM docker.io/library/python:3.12-slim@sha256:2f17fc044b579bab302c2e8054d3a686e2cb9a83de48e70534b94cd8ebbe06a9
=> => resolve docker.io/library/python:3.12-slim@sha256:2f17fc044b579bab302c2e8054d3a686e2cb9a83de48e70534b94cd8ebbe06a9
=> [internal] load build context
=> => transferring context: 65B
=> CACHED [2/5] WORKDIR /app
=> [3/5] COPY requirements.txt .
=> [4/5] RUN pip install --no-cache-dir -r requirements.txt
```

our current step:

[4/5] RUN pip install --no-cache-dir -r requirements.txt

is the slowest part because Python dependencies are being installed for ARM64.

3. Push it

`docker push ghcr.io/valinfernanda/task-management-api:latest`

Then restart: `kubectl rollout restart deployment task-management-api`

And monitor: `kubectl rollout status deployment task-management-api`

```
(.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform\app> kubectl rollout restart deployment task-management-api
deployment.apps/task-management-api restarted
(.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform\app> kubectl rollout status deployment task-management-api
deployment "task-management-api" successfully rolled out
(.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform\app>
```

4. Test /metrics inside the new Pod

`kubectl get pods`

These two are new pods:

```
(.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform\app> kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
task-management-api-869f7976f7-g45z2 1/1     Running   0           35s
task-management-api-869f7976f7-lrxjx 1/1     Running   0           32s
```

Tests FastAPI directly inside the Pod : `kubectl exec -it task-management-api-`

`869f7976f7-g45z2 -- python -c "import urllib.request;`

`print(urllib.request.urlopen('http://127.0.0.1:8000/metrics').read().decode()[:1000])"`

```
(.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform\app> kubectl exec -it task-management-api-869f7976f7-g45z2 -- python -c "import urllib.request; print(urllib.request.urlopen('http://127.0.0.1:8000/metrics').read().decode()[:1000])"
python_gc_objects_collected_total{generation="2"} 0.0
# HELP python_gc_objects_uncollectable_total Uncollectable objects found during GC
# TYPE python_gc_objects_uncollectable_total counter
python_gc_objects_uncollectable_total{generation="0"} 0.0
python_gc_objects_uncollectable_total{generation="1"} 0.0
python_gc_objects_uncollectable_total{generation="2"} 0.0
# HELP python_gc_collections_total Number of times this generation was collected
# TYPE python_gc_collections_total counter
python_gc_collections_total{generation="0"} 114.0
python_gc_collections_total{generation="1"} 10.0
python_gc_collections_total{generation="2"} 0.0
# HELP python_info Python platform information
# TYPE python_info gauge
python_info{implementation="CPython",major="3",minor="12",patch
```

Tests whether the Kubernetes Service can reach the Pod : `kubectl port-forward`

`service/task-management-api 8000:80`

```
(.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform\app> kubectl port-forward service/task-management-api 8000:80
Forwarding from 127.0.0.1:8000 -> 8000
Forwarding from [::1]:8000 -> 8000
```

The easiest way to understand the difference		
Command	What it tests	Traffic path
<code>kubectl exec ...</code>	Pod/FastAPI directly	Pod → localhost:8000
<code>kubectl port-forward service/...</code>	Kubernetes Service → Pod → FastAPI	PC → Service → Pod
<code>curl http://172.169.163.204/metrics</code>	External LoadBalancer	Internet → LoadBalancer → Service → Pod

In other words:

1. `kubectl exec --` Tests FastAPI directly inside the Pod
2. `kubectl port-forward --` Tests whether the Kubernetes Service can reach the Pod
3. External LoadBalancer IP -- Tests the complete external path:

Internet → LoadBalancer → Service → Pod → FastAPI

Now, test through the AKS Service

```
(.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform\app> kubectl get services
NAME                TYPE          CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE
kubernetes           ClusterIP     10.2.0.1      <none>         443/TCP          5d1h
task-management-api  LoadBalancer 10.2.154.242  172.169.163.204 80:31887/TCP     2d2h
```

Our LoadBalancer IP was: 172.169.163.204

Run: `curl http://172.169.163.204/metrics`

```
← → ↻ ⚠ Not secure 172.169.163.204/metrics ☆

# HELP python_gc_objects_collected_total Objects collected during gc
# TYPE python_gc_objects_collected_total counter
python_gc_objects_collected_total{generation="0"} 315.0
python_gc_objects_collected_total{generation="1"} 179.0
python_gc_objects_collected_total{generation="2"} 0.0
# HELP python_gc_objects_uncollectable_total Uncollectable objects found during GC
# TYPE python_gc_objects_uncollectable_total counter
python_gc_objects_uncollectable_total{generation="0"} 0.0
python_gc_objects_uncollectable_total{generation="1"} 0.0
python_gc_objects_uncollectable_total{generation="2"} 0.0
# HELP python_gc_collections_total Number of times this generation was collected
# TYPE python_gc_collections_total counter
python_gc_collections_total{generation="0"} 114.0
python_gc_collections_total{generation="1"} 10.0
python_gc_collections_total{generation="2"} 0.0
# HELP python_info Python platform information
# TYPE python_info gauge
python_info{implementation="CPython",major="3",minor="12",patchlevel="14",version="3.12.14"} 1.0
# HELP process_virtual_memory_bytes Virtual memory size in bytes.
# TYPE process_virtual_memory_bytes gauge
process_virtual_memory_bytes 1.37568256e+08
# HELP process_resident_memory_bytes Resident memory size in bytes.
```

/metrics works now, means **the FastAPI → Docker → GHCR → AKS part is fixed.**

Then we can move on to:

AKS → Prometheus → Grafana

35. Create a ServiceMonitor

The next step is to tell **Prometheus where your FastAPI /metrics endpoint is.**

There are a couple of Kubernetes-native ways to do this. We'll use a **ServiceMonitor**, which is the standard approach with kube-prometheus-stack.

First, we need to make sure our Kubernetes Service has the label that the ServiceMonitor will select.

Run: `kubectl get service task-management-api -o yaml`

Look for:

metadata:

labels:

```
PS C:\Users\Valin\Downloads\cloud-native-devops-platform\app> kubectl get service task-management-api -o yaml
apiVersion: v1
kind: Service
metadata:
  annotations:
    kubernetes.io/last-applied-configuration: |
      {"apiVersion":"v1","kind":"Service","metadata":{"annotations":{},"name":"task-management-api","namespace":"default"},"spec":{"ports":[{"port":80,"protocol":"TCP","targetPort":8000}],"selector":{"app":"task-management-api"},"type":"LoadBalancer"}}
    creationTimestamp: "2026-09-24T12:05:37Z"
  finalizers:
    - service.kubernetes.io/load-balancer-cleanup
  name: task-management-api
```

Our Service currently has no labels, so let's add one specifically for Prometheus discovery.

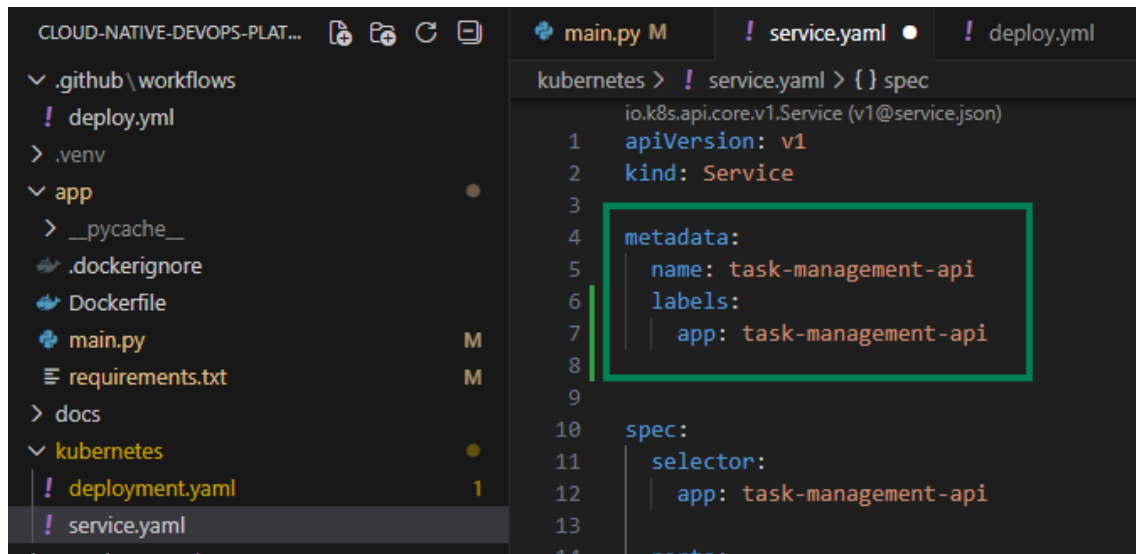
35a. Update service.yaml

Open:

kubernetes/service.yaml

Change the metadata section to:

```
metadata:
  name: task-management-api
  labels:
    app: task-management-api
```



```
CLOUD-NATIVE-DEVOPS-PLAT... main.py M service.yaml deploy.yaml
kubernetes > ! service.yaml > {} spec
1  apiVersion: v1
2  kind: Service
3
4  metadata:
5    name: task-management-api
6    labels:
7      app: task-management-api
8
9
10 spec:
11   selector:
12     app: task-management-api
13
14   ports:
```

We're going to tell Prometheus: "Find the Kubernetes Service with this label, and scrape its /metrics endpoint."

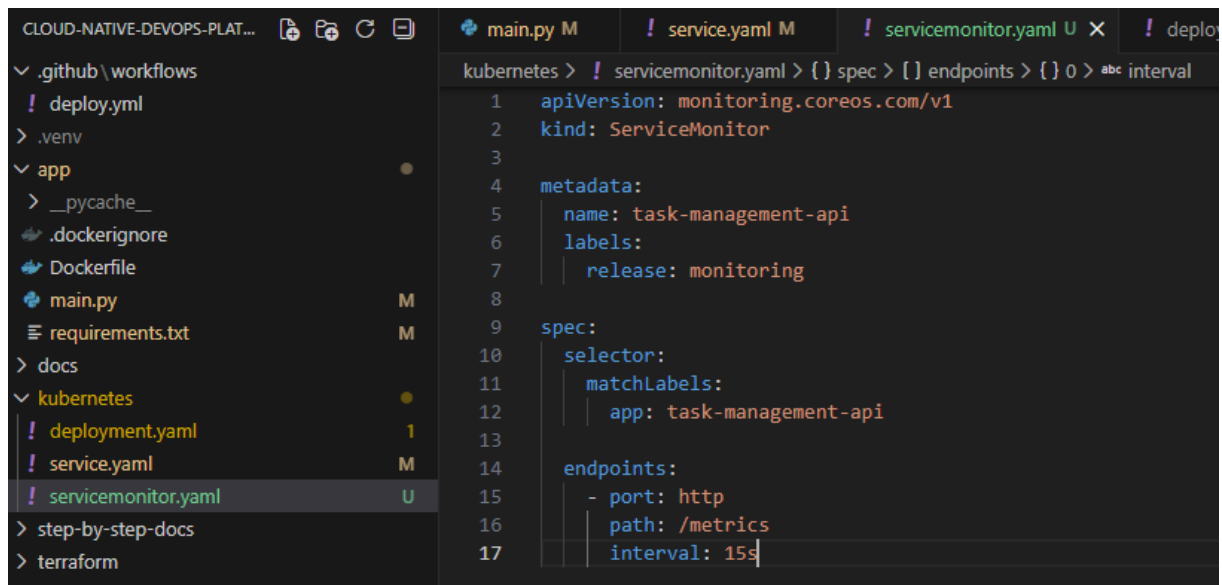
35b. Create kubernetes/servicemonitor.yaml

```
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
```

```
metadata:
  name: task-management-api
  labels:
    release: monitoring
```

```
spec:
  selector:
    matchLabels:
      app: task-management-api
```

```
endpoints:
  - port: http
    path: /metrics
    interval: 15s
```

A screenshot of a code editor with a dark theme. The left sidebar shows a file tree for a project named 'CLOUD-NATIVE-DEVOPS-PLAT...'. The tree includes folders like '.github\workflows', 'docs', 'kubernetes', and 'terraform', along with various files like 'deploy.yml', '.venv', 'app', '__pycache__', '.dockerignore', 'Dockerfile', 'main.py', 'requirements.txt', 'deployment.yaml', 'service.yaml', and 'servicemonitor.yaml'. The 'servicemonitor.yaml' file is selected and open in the main editor. The editor has tabs for 'main.py M', 'service.yaml M', 'servicemonitor.yaml U', and 'deploy...'. The code in the editor is a Kubernetes ServiceMonitor manifest. It starts with 'apiVersion: monitoring.coreos.com/v1' and 'kind: ServiceMonitor'. The 'metadata' section has 'name: task-management-api' and 'labels: release: monitoring'. The 'spec' section has a 'selector' with 'matchLabels: app: task-management-api' and an 'endpoints' list with one entry: 'port: http', 'path: /metrics', and 'interval: 15s'.

```
1 apiVersion: monitoring.coreos.com/v1
2 kind: ServiceMonitor
3
4 metadata:
5   name: task-management-api
6   labels:
7     release: monitoring
8
9 spec:
10   selector:
11     matchLabels:
12       app: task-management-api
13
14   endpoints:
15     - port: http
16       path: /metrics
17       interval: 15s
```

One important issue!!

Our current Service has:

ports:

- protocol: TCP
- port: 80
- targetPort: 8000

but it doesn't have a port name.

Our ServiceMonitor says:

port: http

So Kubernetes needs the Service port to be named http.

Open kubernetes/service.yaml again and change to:

ports:

- name: http
- protocol: TCP
- port: 80
- targetPort: 8000

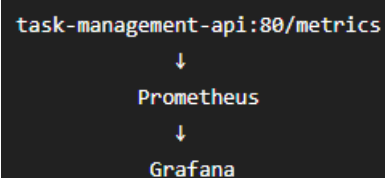
```
main.py M service.yaml M !
kubernetes > ! service.yaml > {} spec
io.k8s.api.core.v1.Service (v1@service.json)
1  apiVersion: v1
2  kind: Service
3
4  metadata:
5    name: task-management-api
6    labels:
7      app: task-management-api
8
9
10 spec:
11   selector:
12     app: task-management-api
13
14   ports:
15     - name: http
16       protocol: TCP
17       port: 80
18       targetPort: 8000
19
20   type: LoadBalancer
21
```

Why this matters

The ServiceMonitor says:

```
Service label → app=task-management-api
Port → http
Path → /metrics
Every → 15 seconds
```

So Prometheus knows exactly where to go:



36. Apply the Service + ServiceMonitor

From our project root, run:

```
kubectl apply -f kubernetes/service.yaml
```

Then: `kubectl apply -f kubernetes/servicemonitor.yaml`

```

● PS C:\Users\Valin\Downloads\cloud-native-devops-platform> kubectl apply -f kubernetes/service.yaml
service/task-management-api configured
● PS C:\Users\Valin\Downloads\cloud-native-devops-platform> kubectl apply -f kubernetes/servicemonitor.yaml
servicemonitor.monitoring.coreos.com/task-management-api created

```

37. Verify the ServiceMonitor

kubectl get servicemonitor

```

● PS C:\Users\Valin\Downloads\cloud-native-devops-platform> kubectl get servicemonitor
NAME                                AGE
task-management-api                78s

```

We've successfully created the Prometheus discovery configuration.

37a. Check Prometheus targets

kubectl port-forward -n monitoring svc/monitoring-kube-prometheus-prometheus 9090:9090

```

● PS C:\Users\Valin\Downloads\cloud-native-devops-platform> kubectl port-forward -n monitoring svc/monitoring-kube-prometheus-prometheus 9090:9090
Forwarding from 127.0.0.1:9090 -> 9090
Forwarding from [::1]:9090 -> 9090

```

Keep that PowerShell window running.

Then open **another PowerShell window** and run:

curl http://localhost:9090/api/v1/targets

```

(.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform> curl http://localhost:9090/api/v1/targets

Security Warning: Script Execution Risk
Invoke-WebRequest parses the content of the web page. Script code in the web page might be run when the page is parsed.
RECOMMENDED ACTION:
  Use the -UseBasicParsing switch to avoid script code execution.

Do you want to continue?

[Y] Yes  [A] Yes to All  [N] No  [L] No to All  [S] Suspend  [?] Help (default is "N"): Y

StatusCode      : 200
StatusDescription : OK
Content          : {"status":"success","data":{"activeTargets":[{"discoveredLabels":{"__address__":"10.0.1.14:8080","__always_scrape_classic_histograms__":"false","__conver
t_classic_histograms_to_nhcb__":"false","__meta...
RawContent       : HTTP/1.1 200 OK

```

Since the port-forward is running, let's check it directly in the **Prometheus UI** — this is easier than parsing the JSON.

37b. Open Prometheus

Go to: **http://localhost:9090/targets**

Look under **Scrape Pools / ServiceMonitor** for something related to:

task-management-api

localhost:9090/targets

Query Alerts Status > Target health

Select scrape pool Filter by target health Filter by endpoint or labels

Hiding pools with no matching targets
Hiding 4 empty pools due to filters or no targets. [Show empty pools](#)

serviceMonitor/default/task-management-api/0 2 / 2 up

Endpoint	Labels	Last scrape	State
http://10.0.1.27:8000/metrics	container="task-management-api" endpoint="http" instance="10.0.1.27:8000" job="task-management-api" namespace="default" pod="task-management-api-869f7976f7-g45z2" service="task-management-api"	2m 24.276s ago 3ms	UP
http://10.0.1.14:8000/metrics	container="task-management-api" endpoint="http" instance="10.0.1.14:8000" job="task-management-api" namespace="default" pod="task-management-api-869f7976f7-lrxjx" service="task-management-api"	2m 28.416s ago 2ms	UP

serviceMonitor/monitoring/monitoring-grafana/0 1 / 1 up

Endpoint	Labels	Last scrape	State
http://10.0.1.15:3000/metrics	container="grafana" endpoint="http-web" instance="10.0.1.15:3000" job="monitoring-grafana" namespace="monitoring"	2m 38.414s ago 30ms	UP

UP means Prometheus is successfully discovering and scraping your FastAPI application's /metrics endpoint.

So our monitoring pipeline is now working end-to-end:

39. Create an API Requests panel in Grafana

Go back to our **AKS Overview** dashboard in Grafana. If the port-forward Grafana is closed already, run this again:

```
kubectrl port-forward -n monitoring svc/monitoring-grafana 3000:80
```

Click:

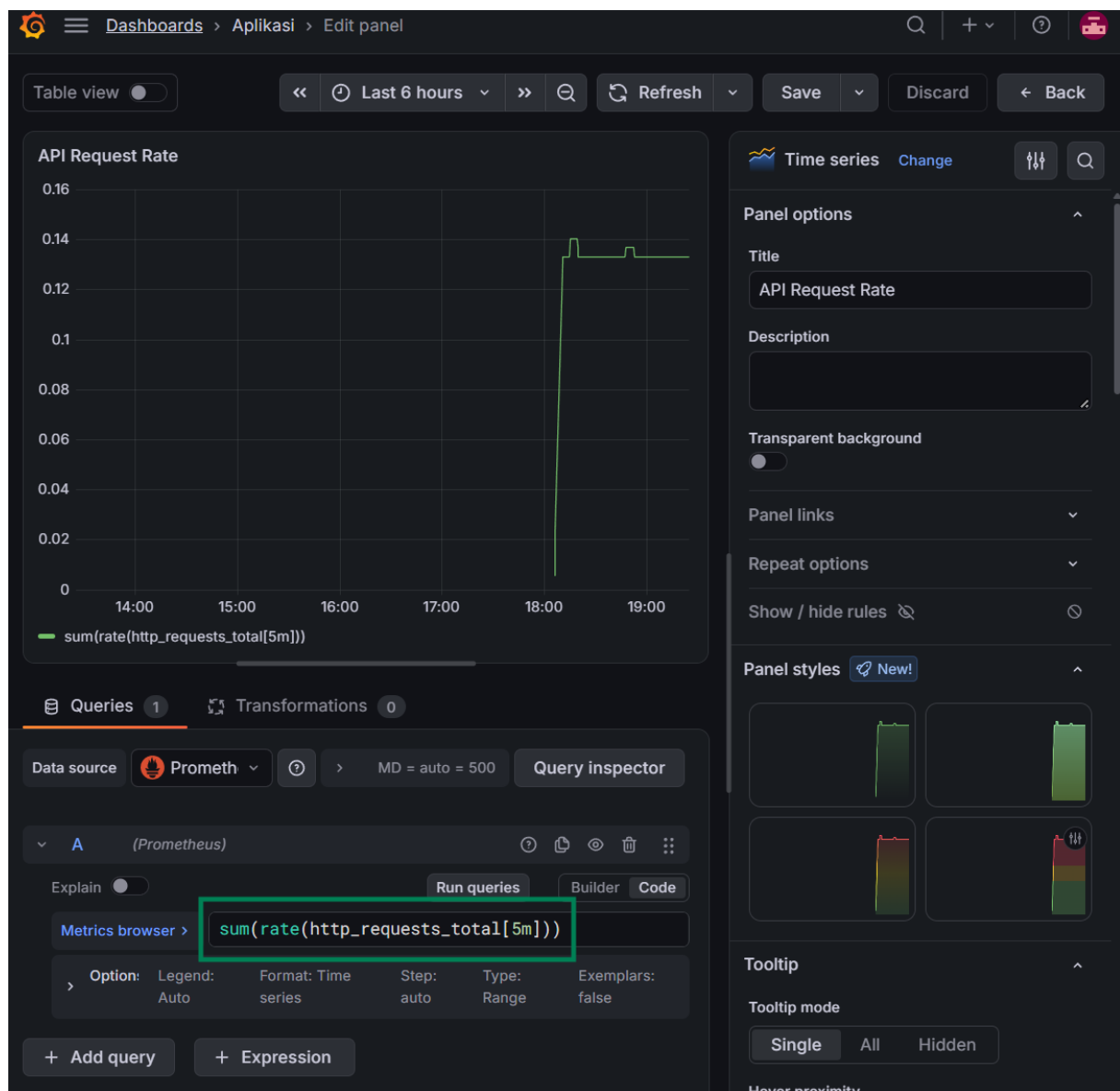
Edit dashboard → Add visualization

For the Prometheus query, use: **sum(rate(http_requests_total[5m]))**

Set the visualization to **Time series**.

Title it: **API Request Rate**

This shows approximately how many HTTP requests per second your FastAPI application is receiving, averaged over the last 5 minutes.

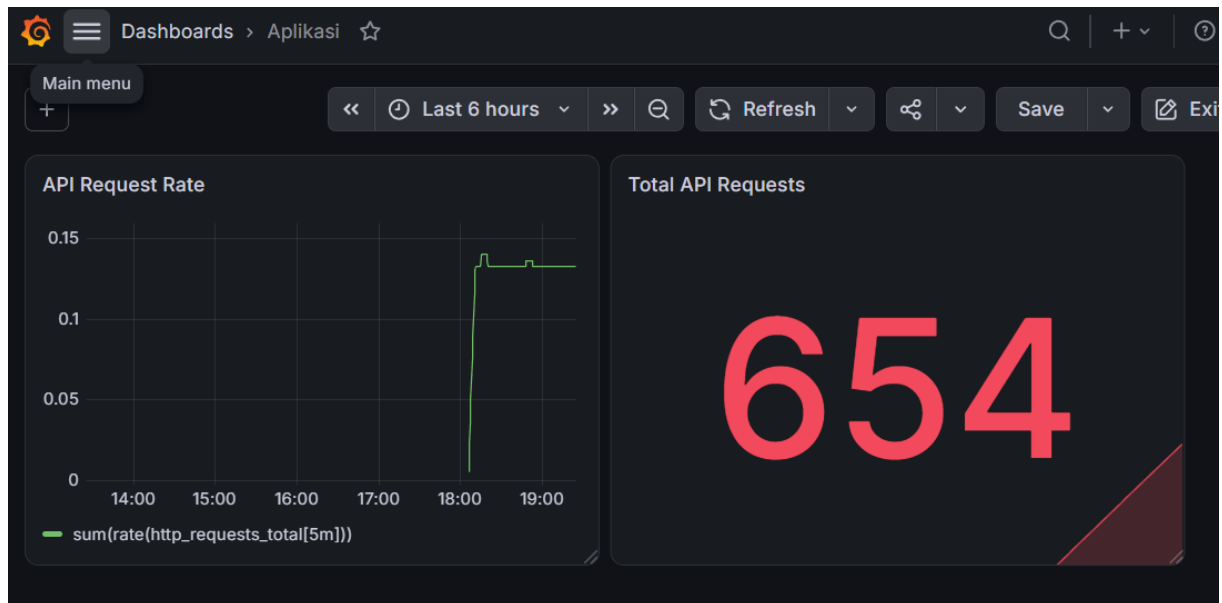


40. Add API Request Count in Grafana

Use this PromQL: **sum(http_requests_total)**

Set:

- **Visualization:** Stat
- **Title:** Total API Requests



41. Add API Response Time in Grafana

Add another visualization and use:

```
histogram_quantile(
  0.95,
  sum(rate(http_request_duration_seconds_bucket[5m])) by (le)
)
```

Set:

- **Visualization:** Time series
- **Title:** API Response Time (p95)
- **Unit:** seconds (s)

The **p95** means approximately: *95% of requests completed within this response time; the slowest ~5% are above it.*

Why do we care about p95?

Because the **average response time can hide slow requests.**

For example:

95 requests → 50 ms

5 requests → 2 seconds

The average might still look reasonable, but 5% of the requests are very slow.

p95 helps answer:

"How quickly do most users get a response, while still accounting for slower requests?"

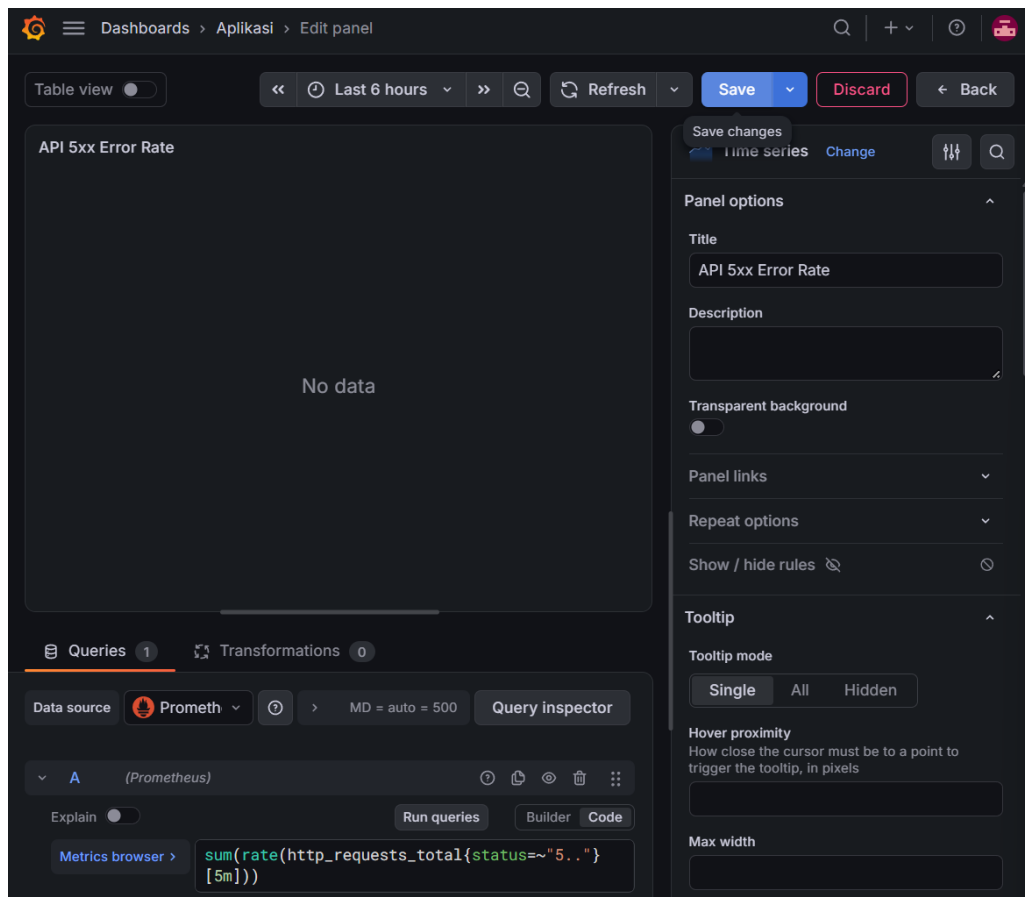
42. Add HTTP Error Rate

```
sum(rate(http_requests_total{status=~"5.."}[5m]))
```

Set:

- **Visualization:** Time series
- **Title:** API 5xx Error Rate
- **Unit:** requests/sec

only shows **5xx responses**. If your API currently has **zero 5xx errors**, the result is empty/zero, so Grafana may appear to have no graph.



43. Test the dashboard with real traffic

We can generate a few requests to your API and then see the metrics move.

In PowerShell, run:

```
kubectrl port-forward service/task-management-api 8000:80
```

Keep that window open.

Then in another PowerShell:

```
curl http://localhost:8000/
```

```
curl http://localhost:8000/health
```

```
curl http://localhost:8000/tasks
```

Run each command a few times.

Then go back to Grafana → AKS Overview and refresh the dashboard.

We should see activity in API Request Rate and Total API Requests.

44. API Availability in Grafana

This is useful because it shows whether Prometheus currently considers your application target reachable.

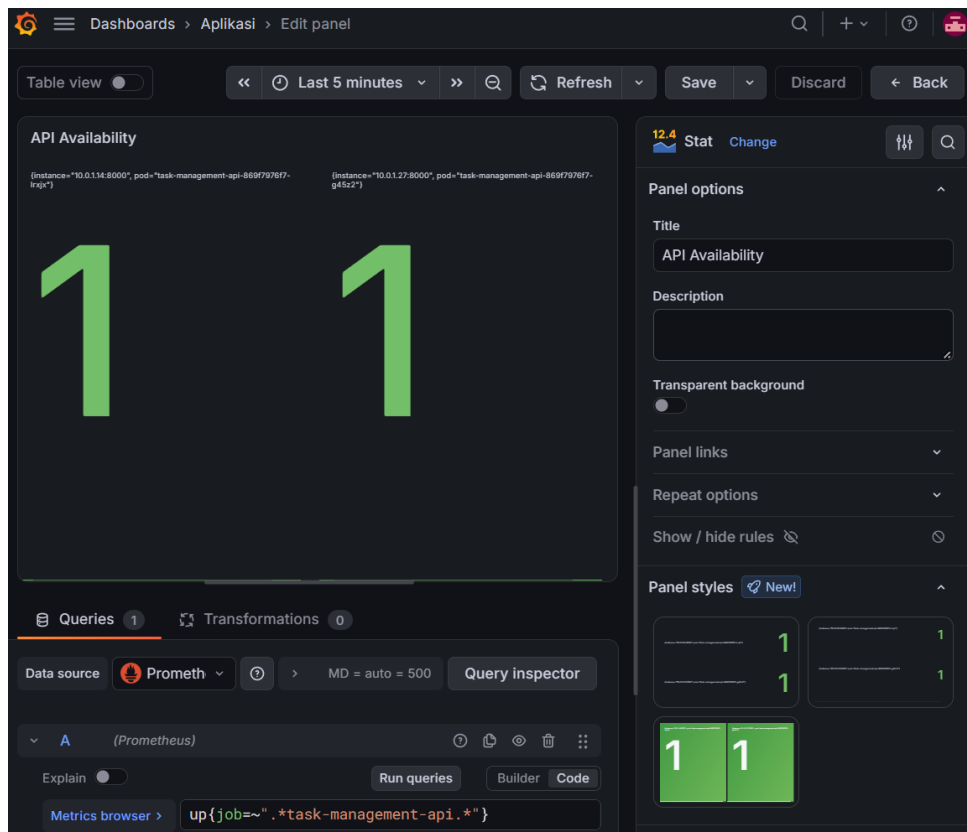
In Grafana → Add visualization, use:

```
up{job=~".*task-management-api.*"}
```

Set:

- Visualization: Stat
- Title: API Availability
- Value mappings:
 - 1 → UP
 - 0 → DOWN

If the query returns 1, your API target is currently being successfully scraped by Prometheus.



Our Grafana dashboard is now having:

Infrastructure

- AKS Node CPU Usage
- AKS Node Memory Usage
- Total Kubernetes Pods
- AKS Node Status

Application

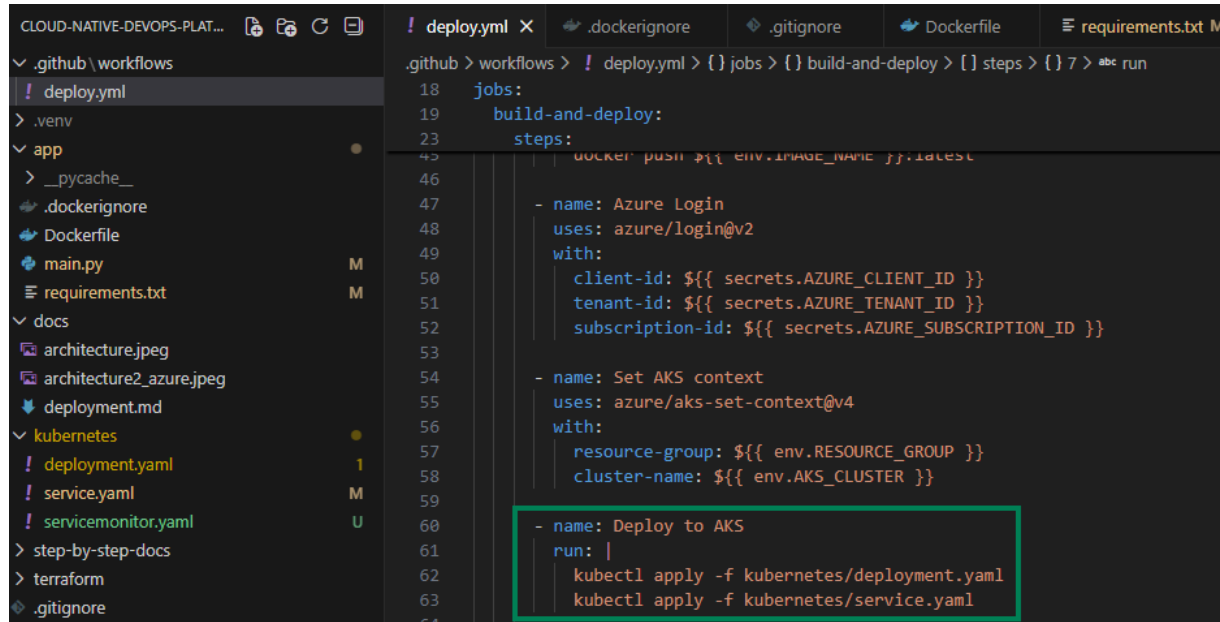
- API Request Rate
- Total API Requests
- API Response Time (p95)
- API 5xx Error Rate
- API Availability

BEFORE WE CONTINUE...

I can see one more thing: our workflow currently deploys only:

kubernetes/deployment.yaml

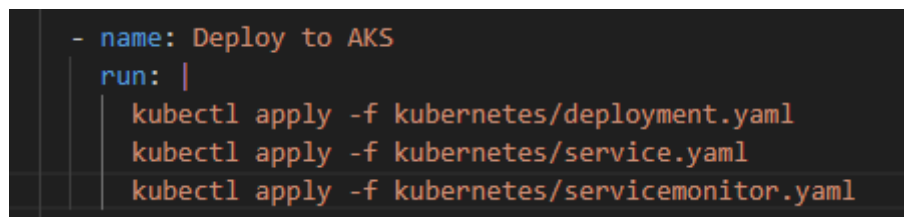
kubernetes/service.yaml



```
18 jobs:
19   build-and-deploy:
20     steps:
21       - name: Azure Login
22         uses: azure/login@v2
23         with:
24           client-id: ${ secrets.AZURE_CLIENT_ID }
25           tenant-id: ${ secrets.AZURE_TENANT_ID }
26           subscription-id: ${ secrets.AZURE_SUBSCRIPTION_ID }
27
28       - name: Set AKS context
29         uses: azure/aks-set-context@v4
30         with:
31           resource-group: ${ env.RESOURCE_GROUP }
32           cluster-name: ${ env.AKS_CLUSTER }
33
34       - name: Deploy to AKS
35         run: |
36           kubectl apply -f kubernetes/deployment.yaml
37           kubectl apply -f kubernetes/service.yaml
```

But we also have: kubernetes/servicemonitor.yaml

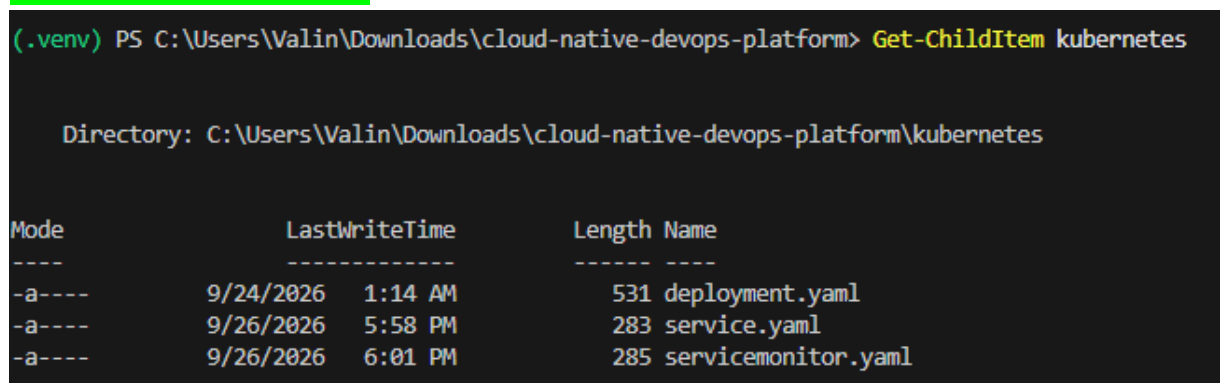
Since our Prometheus monitoring is already working, I'd add that to the CI/CD deployment too. Otherwise a fresh deployment from GitHub Actions doesn't explicitly apply the ServiceMonitor.



```
- name: Deploy to AKS
  run: |
    kubectl apply -f kubernetes/deployment.yaml
    kubectl apply -f kubernetes/service.yaml
    kubectl apply -f kubernetes/servicemonitor.yaml
```

Now.. servicemonitor.yaml has been added

Get-ChildItem kubernetes



```
(.venv) PS C:\Users\Valin\Downloads\cloud-native-devops-platform> Get-ChildItem kubernetes

Directory: C:\Users\Valin\Downloads\cloud-native-devops-platform\kubernetes

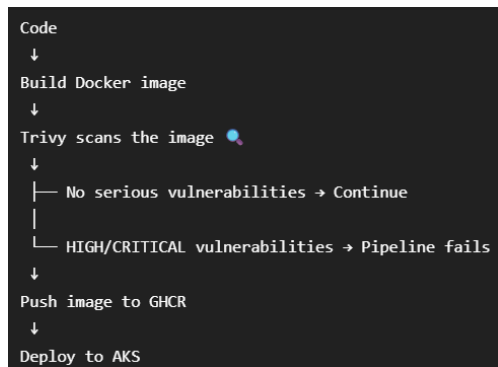
Mode                LastWriteTime         Length Name
----                -
-a----           9/24/2026   1:14 AM             531 deployment.yaml
-a----           9/26/2026   5:58 PM             283 service.yaml
-a----           9/26/2026   6:01 PM             285 servicemonitor.yaml
```

44. Security Scanning

Security scanning means checking the Docker image for known security vulnerabilities before deploying it.

In our project, we use **Trivy** in the GitHub Actions pipeline.

The flow is:



Trivy checks things such as:

- OS packages inside the image
- Python dependencies
- System libraries
- Known vulnerabilities (CVEs)

For example, our image:

ghcr.io/valinfernanda/task-management-api:latest

can be scanned before it is deployed to AKS.

So the main idea is:

We use Trivy to automatically scan our container image for security vulnerabilities as part of the CI/CD pipeline. If HIGH or CRITICAL vulnerabilities are found, the pipeline can fail and prevent the image from being deployed.

This is part of the **DevSecOps** approach because security is integrated directly into the CI/CD pipeline.

It is good so our CI/CD pipeline doesn't blindly deploy an image without checking it for known vulnerabilities.

- name: Scan Docker image with Trivy

uses: aquasecurity/trivy-action@v0.35.0

with:

```
image-ref: ${{ env.IMAGE_NAME }}:${{ github.sha }}
```

```
format: table
```

```
exit-code: 1
```

```
ignore-unfixed: true
```

```
severity: CRITICAL,HIGH
```

Because this process should be done after Build Docker Image and before Push Docker Image to GHCR, add the relevant section above to **deploy.yml**

[exit-code: 1](#) means that if Trivy finds HIGH or CRITICAL vulnerabilities, the GitHub Actions job fails and the image won't be pushed.

The difference is how GitHub Actions chooses which version of the Trivy action to use.

Notes :

[@master](#)

`uses: aquasecurity/trivy-action@master`

This means: use whatever code is currently on the master branch.

So if the Trivy team changes master tomorrow, your workflow will automatically use the new code—even though you didn't change your workflow.

[@v0.35.0](#)

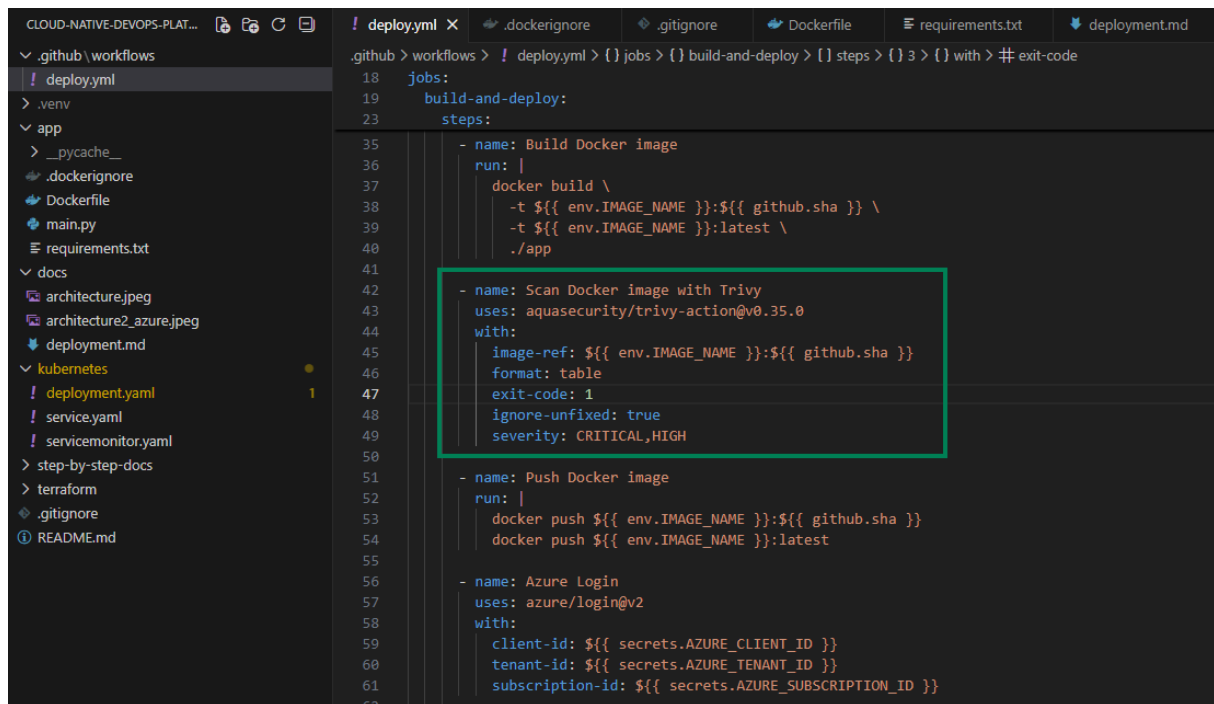
`uses: aquasecurity/trivy-action@v0.35.0`

This means: use the specific 0.33.1 release.

Your workflow will keep using that version until you explicitly update it.

	@master	@v0.35.0
Fixed version	✗	☑
Can change unexpectedly	☑	✗
Reproducible CI	Less reliable	More reliable
Recommended for stable CI	Less ideal	Better

So it is basically about **stability and predictability**. It doesn't change what Trivy does; it makes sure your CI doesn't silently start using a different version of the **Trivy GitHub Action**.



```
18 jobs:
19   build-and-deploy:
20     steps:
21       - name: Build Docker image
22         run: |
23           docker build \
24             -t ${ env.IMAGE_NAME }:${ github.sha } \
25             -t ${ env.IMAGE_NAME }:latest \
26             ./app
27       - name: Scan Docker image with Trivy
28         uses: aquasecurity/trivy-action@v0.35.0
29         with:
30           image-ref: ${ env.IMAGE_NAME }:${ github.sha }
31           format: table
32           exit-code: 1
33           ignore-unfixed: true
34           severity: CRITICAL,HIGH
35       - name: Push Docker image
36         run: |
37           docker push ${ env.IMAGE_NAME }:${ github.sha }
38           docker push ${ env.IMAGE_NAME }:latest
39       - name: Azure Login
40         uses: azure/login@v2
41         with:
42           client-id: ${ secrets.AZURE_CLIENT_ID }
43           tenant-id: ${ secrets.AZURE_TENANT_ID }
44           subscription-id: ${ secrets.AZURE_SUBSCRIPTION_ID }
```

Now commit and push

```
git add .github/workflows/deploy.yml
```

```
git commit -m "ci: add container security scanning"
```

```
git push
```

After pushing, don't change anything else yet. Go to GitHub → Actions → Build and Deploy and check the new workflow run.

After error some time and have tried to fix it, finally green:

valinfernanda / cloud-native-devops-platform

Code Issues Pull requests Actions Projects Wiki Security and quality Insights Settings

Actions

New workflow

All workflows

Build and Deploy

Management

- Caches
- Attestations
- Runners
- Usage metrics
- Performance metrics

Build and Deploy

deploy.yml

Filter workflow runs

8 workflow runs

	Event	Status	Branch	Actor
✓ upload the rest or file	Build and Deploy #8: Commit 828338b pushed by valinfernanda	main	2 minutes ago	57s
✗ fix: update Trivy action version	Build and Deploy #7: Commit 71bd700 pushed by valinfernanda	main	5 minutes ago	40s
✗ fix: update Trivy action version	Build and Deploy #6: Commit 32353fd pushed by valinfernanda	main	18 minutes ago	22s
✗ ci: add container security scanning	Build and Deploy #5: Commit bb2888e pushed by valinfernanda	main	24 minutes ago	8s
✓ docs: update project documentation	Build and Deploy #4: Commit 210e048 pushed by valinfernanda	main	Sep 26, 8:21 PM GMT+2	34s
✓ Fix Kubernetes manifest paths	Build and Deploy #3: Commit c8f23af pushed by valinfernanda	main	Sep 25, 8:53 PM GMT+2	49s

Login Azure CLI : az login --tenant "62ab525c-f89b-49ac-94e1-257f270456d4"

Summary

All jobs

build-and-deploy

Run details

Usage

Workflow file

build-and-deploy

succeeded 3 minutes ago in 54s

Search logs

> Login to GHCR

> Build Docker image

> Scan Docker image with Trivy

> Push Docker image

> Azure Login

1 ▶ Run aquasecurity/trivy-action@v0.35.0

21 ▶ Install Trivy

22 ▶ Run aquasecurity/setup-trivy@e6c2c5e321ed9123bda567646e2f96565e34abe1

33 ▶ Binary dir

42 ▶ Restore Trivy binary from cache

58 ▶ Checkout install script

131 ▶ Install Trivy

146 ▶ Add Trivy binary to \$GITHUB_PATH

154 ▶ Get current date

155 ▶ Run echo "date=\$(date +%Y-%m-%d)" >> \$GITHUB_OUTPUT

162 ▶ Restore DB from cache

163 ▶ Run actions/cache@0400d5f644dc74513175e3cd8d07132dd4860809

176 (node:2608) [DEP0040] DeprecationWarning: The `punycode` module is deprecated. Please use a userland alternative instead.

177 (Use `node --trace-deprecation ...` to show where the warning was created)

178 Cache not found for input keys: cache-trivy-2026-09-27, cache-trivy-

179 ▶ Set GitHub Path

180 ▶ Run echo "\$GITHUB_ACTION_PATH" >> \$GITHUB_PATH

188 ▶ Clear Trivy Envs file

189 ▶ Run rm -f trivy_envs.txt

197 ▶ Set Trivy environment variables

198 ▶ Run # Note: There is currently no way to distinguish between undefined variables and empty strings in GitHub Actions.

270 ▶ Run Trivy

359 ▶ Remove Trivy Envs file

So Trivy successfully installed, loaded its vulnerability database, and scanned our Docker image.

kubectl config current-context

Login Azure CLI : az login --tenant "62ab525c-f89b-49ac-94e1-257f270456d4"